x86 Architecture

William Bland

March 4, 2026

Contents

1 Introduction **3**

2 x86 architecture **3**
2.1 History of x86 . . . 3
2.2 Operating modes . . . 4
2.3 Multiprocessor . . . 4

3 Memory **4**
3.1 Registers . . . 5
3.1.1 General Purpose Registers . . . 5
3.1.2 EFLAGS . . . 5
3.1.3 Instruction Pointer . . . 6
3.1.4 Segment Registers . . . 6
3.1.5 Control Registers . . . 7
3.1.6 Model Specific Registers . . . 8
3.2 Cache . . . 9
3.2.1 Cache coherency protocol . . . 9
3.2.2 Cache Placement Policy . . . 10
3.2.3 Cache misses . . . 11
3.2.4 Memory Types . . . 11
3.2.4.1 MTRRs . . . 12
3.2.4.2 Page Attribute Table . . . 13
3.2.4.3 Effective Memory Type . . . 14
3.3 Memory addressing . . . 15
3.4 Segmentation . . . 15
3.4.1 Real Mode . . . 15
3.4.2 Protected Mode . . . 15
3.4.3 Long Mode . . . 16
3.4.4 Segment Selectors . . . 16
3.4.5 Segment Descriptors . . . 16
3.4.6 Segment Descriptor Tables . . . 19
3.5 Paging . . . 19
3.5.1 Enabling Paging . . . 20
3.5.2 Paging Structures . . . 20
3.5.3 Translation Lookaside Buffer . . . 23
3.6 Virtual Memory . . . 23

4 Task Management **24**
4.1 Segment Descriptors . . . 24
4.2 16-bit TSS . . . 25
4.3 32-bit TSS . . . 26
4.4 64-bit mode . . . 27
4.5 Call Gates . . . 27

5 Instructions **28**

6 Interrupts **33**
6.1 Sources of interrupts . . . 34
6.2 Interrupt Handling . . . 35
6.2.1 Protected Mode Handling . . . 35
6.2.2 IA-32e Mode Handling . . . 36
6.2.3 Error Codes . . . 36
6.3 PIC . . . 36
6.4 APIC . . . 38
6.4.1 Priority . . . 40
6.4.2 Errors . . . 41
6.4.3 Interprocessor Interrupts . . . 41
6.4.4 Timer . . . 42
6.4.5 Spurious Interrupt . . . 43
6.5 Interrupt Descriptor Table . . . 43
6.6 Real Mode Interrupts . . . 44
6.7 BIOS Interrupts . . . 45
6.8 Syscalls . . . 45

7 Appendix **46**
7.1 Bash scripts . . . 46
7.2 Hello world (Linux) . . . 47
7.3 Difference (Linux) . . . 48
7.4 Cat (Linux) . . . 49
7.5 Hello world (Baremetal) . . . 51
7.6 Kernel (Baremetal) . . . 52

8 References **63**

9 Acknowledgements **64**

1 Introduction

This document contains the basics of the x86 architecture, and could be used as an introduction into the x86 architecture. It assumes you have some knowledge of programming and computer science. Additionally it does summarise some sections (such as instructions) to keep them concise, however it should sufficient to write basic x86 assembly programs (some examples can be seen in the appendix) or even a simple bare-metal program/kernel (can also be seen in the appendix); all code written is assembled with NASM and linked with ld. Furthermore, this document is mainly based off "`Intel® 64 and IA-32 Architectures Software Developer's Manuals`", hence may have some content specific to the newest Intel CPUs.

2 x86 architecture

Processors run by sequentially executing binary instructions (machine code), assembly code is a human readable version of machine code which can be converted (assembled) into machine code using an assembler. Every processor has an architecture, this architecture describes how the processor executes machine code (therefore different architectures execute machine code differently, so have different machine code and assembly languages). For multiple processors to be able to execute the same machine code they must have the same architecture, some popular architectures include ARM, PowerPC, RISC-V and x86. x86 is an example of a Complex Instruction Set Computer (CISC), so it uses a large set of instructions that can perform multiple operations in a single command. This is in opposition to Reduced Instruction Set Computer (RISC) which uses a smaller set of simple instructions. CISC (x86) is often more powerful than RISC due to its larger instruction set (allowing what would have been multiple RISC instructions to be ran as a single instruction), however this uses a lot more power therefore CISC (x86) is mainly used for desktop computers while RISC is mainly used for embedded and mobile devices.

2.1 History of x86

Below is a list of a few of the notable x86 processors released by Intel. Furthermore, during the development of these processors and more recent processors, there were also significant improvements such as instruction pipelining, performance and features added such as SIMD and virtualisation.

- The first x86 processors were the 8086 and 8088 which were 16-bit. Through the use of segmentation, it allowed a 20-bit address bus so was able to address 2^{20} Bytes of memory (1MB).
- The Intel 286 processor introduced protected mode by uses the segment registers as selectors into descriptor tables which describe how a segment can be used (so the OS can ensure each process only accesses memory it has the permission to). Furthermore the descriptors provide a 24-bit base address so can address up to 16MB of memory.
- The Intel 386 processor was the first 32-bit processor, introducing 32-bit registers and paging.
- The Intel 486 introduced an integrated x87 FPU (previously FPUs were separate components).
- The Intel Pentium 4 introduced 64-bit extensions.

[1, Chapter 2.1]

2.2 Operating modes

x86 processors have several different operating modes that they can run in, these operating modes can change the addressing mode, available instruction set and registers.

- **Real Mode[5]**:
  Also called real address mode, all addresses always correspond to physical (real) memory addresses; memory addresses are 20 bit allowing for 1MiB of memory, however there is no memory protection. And all x86 CPU's start in real mode for backwards compatibility and start executing from address FFFF:0000
- **Protected Mode[6]**:
  Also called protected virtual address mode, it allows the use of segmentation, virtual memory, paging and safe multi-tasking. To enter protected mode the software must setup a segment descriptor table and enable the Protection Enable (PE) bit in CR0
- **Long Mode[7]**:
  The 64-bit OS can access 64-bit registers and instructions. 64-bit programs run in a sub-mode called 64-bit mode, and 32-bit and 16-bit protected mode programs run in another sub-mode called compatibility mode, however real mode and virtual 8086 mode programs cannot run natively in long mode
- **Unreal Mode[8]**:
  A variant of real mode which allows access to 4GiB of memory
- **System Management Mode[9]**:
  Used by firmware (BIOS/UEFI) for power and hardware control. It is entered through a SMI (System Management Interrupt), and executes code in an address space isolated from other operating modes
- **Virtual 8086 Mode[10]**:
  Also called virtual real mode, it allows real mode processes to run in protected mode. It is a hardware virtualisation technique and uses the 20-bit segmentation that real mode uses but is subject to the protected mode's memory paging

2.3 Multiprocessor

In multiprocessor (multithreaded) environments, the processor is able to execute processes in parallel. This can lead to race conditions if not handled correctly.
x86 processors store writes to memory in a store buffer. It allows the processor to continue execution without having to wait for a write to cache/main memory to complete. The store buffer is flushed when: a LOCK or a MFENCE/SFENCE is used, an I/O instruction executes or an exception/interrupt is generated [3, Chapter 13.10].

3 Memory

Processors operate on digital signals which can either be high or low (interpreted in binary as a 1 or a 0), a single 1 or 0 is a bit.

Data sizes:

Name	Unit	Bits	Bytes
bit	b	1	1/8
nibble	-	4	1/2
byte	B	8	1
word	-	16	2
dword (double word)	-	32	4
qword (quad word)	-	64	8

All memory addresses point to a unique byte in memory therefore a multiple byte integer (eg 64-bit) could be stored in two different orderings (its endianness). In Little-Endian (LE) ordering the least significant byte occupies the smaller address space, while in Big-Endian (BE) ordering the most significant byte occupies the smaller address space. x86 and most other processors use LE while data transmitted over networks uses BE.

[1, Chapter 1.3.1]

3.1 Registers

Resisters store a small amount of memory directly accessible to instructions.

3.1.1 General Purpose Registers

General purpose registers are the most commonly accessed registers, they are used for a variety of uses such as arithmetic, logic and memory addressing. Most instructions require a register to be specified while some may implicitly use a specific register (such as stack operations modifying the stack pointer register). The registers on the same line in the table overlap in memory (eg the 32-bit register is located in the lower half of the 64-bit register). Furthermore both the lower and higher 8-bit registers exist within the 16-bit registers.

64-bit	**32-bit**	**16-bit**	**Higher 8-bit**	**Lower 8-bit**	**Description**
RAX	EAX	AX	AH	AL	Accumulator
RBX	EBX	BX	BH	BL	Base
RCX	ECX	CX	CH	CL	Counter
RDX	EDX	DX	DH	DL	Data (often extends the accumulator)
RSI	ESI	SI	-	SIL	Source index for string operations
RDI	EDI	DI	-	DIL	Destination index for string operations
RSP	ESP	SP	-	SPL	Stack pointer
RBP	EBP	BP	-	BPL	Base pointer (for stack frames)
R8	R8D	R8W	-	R8B	General purpose register
R9	R9D	R9W	-	R9B	General purpose register
R10	R10D	R10W	-	R10B	General purpose register
R11	R11D	R11W	-	R11B	General purpose register
R12	R12D	R12W	-	R12B	General purpose register
R13	R13D	R13W	-	R13B	General purpose register
R14	R14D	R14W	-	R14B	General purpose register
R15	R15D	R15W	-	R15B	General purpose register

Registers 8-15, SIL, DIL, SPL and BPL are only available in 64-bit code since they were added in the 64-bit extensions. Furthermore, 64-bit code cannot access a higher 8-bit register and a register added in the 64-bit extensions in the same instruction.

[1, Chapter 3.4.1]

3.1.2 EFLAGS

The EFLAGS register is a 32-bit register that contains: status flags, a control flag, and system flags. It cannot be directly accessed, instead it can be accessed via instructions such as PUSHFD and POPFD. In IA-32e mode the EFLAGS register is extended to the 64-bit RFLAGS register, the additional 32-bits are all reserved.

The following table describes the flags in the EFLAGS register (unlisted bits are reserved):

Bits	Type	Label	Description
0	Status	CF	Carry flag
2	Status	PF	Parity flag
4	Status	AF	Auxiliary flag
6	Status	ZF	Zero flag
7	Status	SF	Sign flag
8	System	TF	Trap flag
9	System	IF	Interrupt enable flag
10	Control	DF	Direction flag
11	Status	OF	Overflow flag
13:12	System	IOPL	I/O privilege level
14	System	T	Nested task flag
16	System	F	Resume flag
17	System	M	Virtual 8086 mode flag
18	System	C	Alignment check
19	System	IF	Virtual interrupt flag
20	System	IP	Virtual interrupt pending
21	System	D	Identification flag

If the program is able to set/clear the identification flag then the processor supports the CPUID instruction.

[1, Chapter 3.4.3]

3.1.3 Instruction Pointer

The Instruction Pointer register contains a pointer to the next instruction to be executed. It cannot be directly accessed by software, it is controlled by instructions such as JMP, CALL, RET etc. The only way to read the register is to execute the `CALL` instruction and read it from the stack, it can be indirectly loaded by modifying the return address on the stack then executing `RET` or `IRET`. 64-bit mode supports the use of RIP-relative addressing where the effective address is determined by adding a displacement to the RIP of the next instruction, this allows support for Position Independent Executables (PIE).

Name	Bits
IP	16
EIP	32
RIP	64

[1, Chapter 3.5]

3.1.4 Segment Registers

Segment registers are 16-bit registers which are used for memory segmentation. The segment registers are:

Register	Description
CS	Code segment
DS	Data segment
SS	Stack segment
ES	Extra segment (for string operations)
FS	General purpose segment (Since i386)
GS	General purpose segment (Since i386)

Furthermore, each segment register contains a hidden 64-bit register which is used as a segment descriptor cache.

[3, Chapter 3.4.3]

3.1.5 Control Registers

Control registers contain flags that control the operation of the CPU (such as enabling specific features). Control registers not listed in the table below such as CR1 and bits not listed are reserved and should be 0 (writing to a reserved register results in a #UD exception, setting a reserved bit in CR0[0:31] is ignored, setting a reserved bit in CR0[32:63], CR4 or CR3[MAXPHYADDR:63] causes a #GP). When protection is enabled, control registers can only be written to at CPL 0, however CR2 is writable by software. On a 64-bit processor, writing to a control register outside of 64-bit mode sets the upper 32 bits to 0, however when transitioning out of IA-32e mode, the registers retain their upper 32 bits. CR8 is only available in 64-bit mode, and CR4[32:63] only affects IA-32e mode and can only be changed in IA-32e mode.

The following table is a list of the control registers and their flags:

Bits	Label	Description
CR0 - Contains system control flags that control operating mode and states of the processor		
0	PE	Protected Mode Enable
1	MP	Monitor co-processor
2	EM	x87 FPU Emulation
3	TS	Task switched
4	ET	Extension type
5	NE	Numeric error
16	WP	Write protect
18	AM	Alignment mask
29	NW	Not-write through
30	CD	Cache disable
31	PG	Paging
CR2 - Contains linear address that caused a page fault		
0:31(63)	PFLA	Page Fault Linear Address
CR3 - Contains the physical address of the base of the paging-structure hierarchy and two flags		
3	PWT	Page-level Write-Through (not used in PAE or if bit CR4.PCIDE = 1 in long mode)
4	PCD	Page-level Cache Disable (not used in PAE or if bit CR4.PCIDE = 1 in long mode)
12:31(63)	PDBR	Page Directory Base Register (PAE = Base of PDPT, long = base of PML4T/PML5T)
CR4 - Contains flags that enable several architectural extentions		
0	VME	Virtual 8086 Mode Extensions
1	PVI	Protected-mode Virtual Interrupts
2	TSD	Time Stamp Disable
3	DE	Debugging Extensions
4	PSE	Page Size Estension
5	PAE	Physical Address Extension
6	MCE	Machine Check Extension
7	PGE	Page Global Enabled
8	PCE	Performance-Monitoring Counter enable
9	OSFXSR	Operating System Support for FXSAVE and FXRSTOR Instructions
10	OSXMMEXCPT	Operating System Support for Unmasked SIMD Floating-Point Exceptions
11	UMIP	User-Mode Instruction Prevention (if set, #GP on SGDT, SIDT, SLDT, SMSW, and STR instructions when CPL > 0)

Bits	Label	Description
12	LA57	57-bit linear addresses (if set, the processor uses 5-level paging otherwise it uses 4-level paging)
13	VMXE	Virtual Machine Extensions Enable
14	SMXE	Safer Mode Extensions enable
16	FSGSBASE	Enables the instructions: RDFSBASE, RDGSBASE, WRFSBASE and WRGSBASE
17	PCIDE	PCID Enable
18	OSXSAVE	XSAVE and Processor Extended States Enable
20	SMEP	Supervisor Mode Execution Protection Enable
21	SMAP	Supervisor Mode Access Prevention Enable
22	PKE	Protection Key Enable
23	CET	Control-flow Enforcement Technology
24	PKS	Enable Protection Keys for Supervisor-Mode Pages
CR8 - Provides access to the Task Priority Register (TPR)		
0:3	TPL	Task Priority Level

[3, Chapter 2.5]

3.1.6 Model Specific Registers

Model Specific Registers (MSRs) are registers which may not be present on all processors, some are added for testing features and may not be present in future processors, others may provide unique functionality for the processor (eg if the processor were made to be used in a non-standard environment). MSRs are 64-bits in size and identified with a unique 32-bit integer stored in ecx. MSRs are witten to using the `WRMSR` instruction, it writes EAX:EDX to the register; MSRs are read from using the `RDMSR` instruction, it reads from the register into EAX:EDX.

[3, Chapter 11.4]

The most commonly used MSR is the Extended Feature Enable Register (EFER, sometimes IA32_EFER), its address is 0xC000_0080 and its contents are listed in the table below:

Bit	Label	Description
0	SCE	Enables SYSCALL/SYSRET instructions in 64-bit mode
8	LME	Enabled IA-32e mode
10	LMA	Indicates IA-32e mode is active when set
11	NXE	Execute disable bit enable

[4, Page 2-87]

Various other MSR addresses used in this document (their use is described where they are used):

MSR	Address
IA32_MTRR_DEF_TYPE	0x02FF
IA32_MTRR_FIX64K_00000	0x0250
IA32_MTRR_FIX16K_80000	0x0258
IA32_MTRR_FIX16K_A0000	0x0259
IA32_MTRR_FIX4K_C0000	0x0268
IA32_MTRR_FIX4K_C8000	0x0269
IA32_MTRR_FIX4K_D0000	0x026A
IA32_MTRR_FIX4K_D8000	0x026B
IA32_MTRR_FIX4K_E0000	0x026C
IA32_MTRR_FIX4K_E8000	0x026D
IA32_MTRR_FIX4K_F0000	0x026E
IA32_MTRR_FIX4K_F8000	0x026F
IA32_MTRR_PHYSBASEn	0x0202 + 2n
IA32_MTRR_PHYSMASKn	0x0201 + 2n
IA32_PAT	0x0227
IA32_FS_BASE	0xC000_0100
IA32_GS_BASE	0xC000_0101

[4]

3.2 Cache

Cache is a relatively small amount of memory placed within the processor. It is extremely fast SRAM (as opposed to the slower main memory DRAM). There are typically several layers of cache, with each layer getting larger in capacity and slower to access:

1. L1d:
   Extremely fast data cache
2. L1i:
   Extremely fast instruction cache
3. L2:
   Very fast cache
4. L3:
   Fast cache (shared between cores)

3.2.1 Cache coherency protocol

Cache coherency protocols ensure the caches on difference processors (cores) store the most recent, correct copy of data. x86 uses MSI-based protocols, such as MESI, MOESI and MESIF (with the latter two being used in newer AMD and Intel processors respectively); MSI-based protocols are a type of write-invalidate snooping protocol, this means that a write on one cache invalidates all other cache. Each cache line is stored with a state, when a cacheline is accessed (read miss or write), its state is updated and it sends a message to all other processors on a bus. The cache controllers snoops the bus, if a message involves a cacheline in its cache, its state is updated to reflect the message.

The accesses to cache which affect state are:

1. PR - current Processor Reads (cache miss)
2. PW - current Processor Writes
3. BR - Bus Read (cache miss, another processor reads)
4. BW - Bus Write (another processor writes)

The state transitions are defined by a finite state machine, the invalid state is considered as a cache miss and uncached items can be considered to be in the invalid state.

State	Data	Responds to BR	Description
Modified	Dirty	Yes	Only copy of the data
Exclusive	Clean	Yes	Only copy of the data
Shared	Clean*	No**	Multiple copies of the data
Invalid	Invalid	No	The cache line does not hold a valid copy of the data
Owned	Dirty	Yes	Multiple copies of the data (all others are shared)
Forward	Clean	Yes	Multiple copies of the data (all others are shared)

* Dirty if another cache is in the Owned state.
** Will never respond (provide the cached data) if another cache is Owned or Forward, otherwise depending on implementation either a single Shared cache will respond, they all respond or none respond.
Owned and Modified states write to main memory (or next-level cache) when switching to a clean state or when evicted from cache. Furthermore, states with the only copy of cache (Modified and Exclusive) do not have to send a bus message when writing to its cache.
The following finite state machines represent the transitions between the states of their respective protocols (MSI has been included since it is the foundation of the protocols but is not used in x86).

	Access			
State	**PR**	**PW**	**BR**	**BW**
MSI transitions [11]				
M	M	M	S	I
S	S	M	S	I
I	S	M	I	I
MESI transitions [12]				
M	M	M	S	I
E	E	M	S	I
S	S	M	S	I
I	*	M	I	I
MOESI transitions [13]				
M	M	M	O	I
O	O	M	O	I
E	E	M	S	I
S	S	M	S	I
I	*	M	I	I
MESIF transitions [14]				
M	M	M	S	I
E	E	M	S	I
S	S	M	S	I
I	**	M	I	I
F	F	M	S	I

* Transitions to E if exclusive or S if shared.
** Transitions to E if exclusive or F if shared.

3.2.2 Cache Placement Policy

Cache placement policies determine which cacheline to place a block of memory, it is required for efficient lookup and utilisation of cache.

There are three main cache placement policies:

- Direct:
  Each physical address is split into three parts: tag, block and offset. The block is used to identify a specific cacheline, each cacheline stores a tag and the tags are compared, if equal its considered a cache hit else it is a cache miss. On a hit, the offset is used to retrieve a byte from the cacheline. On a miss, the new data is fetched from main memory and replaces the cacheline.

- Associative:
  Each physical address is split into two parts: tag and offset. Every cacheline is checked in parallel for if the tags are equal. On a hit, the offset is used to retrieve a byte from the cacheline. On a miss, the new data is fetched from main memory and replaces a cacheline usually identified by a LRU algorithm (Least Recently Used).

- N-Way Set-Associative:
  Each physical address is split into three parts: tag, block and offset. The block is used to identify a specific set (each set stores N cachelines). Each cacheline in the set is checked in parallel for if the tags are equal. On a hit, the offset is used to retrieve a byte from the cacheline. On a miss, the new data is fetched from main memory and replaces a cacheline within the set usually identified by a LRU algorithm.

The Following table summarises the pros and cons of each policy:

Policy	**Pros**	**Cons**
Direct	Fast (spacial locality)	Cache lines often get evicted if they have the same block
Associative	Cache lines only get evicted if not used (temporal locality)	Slow/Expensive
Set-Associative	Faster than Associative with similar positives	Slower than Direct and Conflict misses can occur

[15]

Most x86 processors use N-Way Set-Associative caches, with the more performant processors using larger set sizes.

3.2.3 Cache misses

A cache miss occurs when valid data does not exist in the processor's cache, requiring a read from main memory. The first three types of cache miss are commonly known as the three C's.

1. Compulsory (cold) miss:
   Occurs when accessing memory for the first time (so was never cached)

2. Capacity miss:
   Occurs when cache is full and a cacheline was evicted but later used

3. Conflict miss:
   Occurs when multiple cache lines are mapped to the same location in cache, causing one to get evicted so if read again later it is a miss

4. Coherence miss:
   The (write-invalidate) cache coherency protocol (eg MESI) invalidated the cacheline because it was written in another processor

[16]

3.2.4 Memory Types

Memory types define the caching and write back rules for a region of memory.

Memory Type	Cacheable	Write-Back Cacheable	Allows Speculative Reads	Memory Ordering Model
Uncacheable (UC)	No	No	No	Strong Ordering
Write Combining (WC)	No	No	Yes	Weak Ordering
Write Through (WT)	Yes	No	Yes	Speculative Processor Ordering
Write Back (WB)	Yes	Yes	Yes	Speculative Processor Ordering
Write Protected (WP)	Reads	No	Yes	Speculative Processor Ordering

Additional notes for the memory types:

- UC: Useful for memory-mapped IO, worst performance.
- WC: Writes may be delayed and combined in the write combining buffer (WC buffer) to reduce memory accesses.
- WT: Writes go to main memory (updating any existing cache lines), write combining is allowed.
- WB: Writes go to cache, write combining is allowed, best performance.
- WP: Writes go to main memory and invalidate all cache lines.

[3, Chapter 13.3]

3.2.4.1 MTRRs

Memory Type Range Registers (MTRRs) are used to define ranges in physical memory, various MSRs specify the memory type in each range.

Memory types that can be encoded with MTRRs:

Encoding	Memory Type
00	UC
01	WC
02	Rsvd.
03	Rsvd.
04	WT
05	WP
06	WB
07-FF	Rsvd.

The IA32_MTRR_DEF_TYPE MSR sets the default memory type for memory regions not specified by an MTRR and is also used to enable MTRRs.

Bits	Use
7:0	Default memory type
10	Fixed-range MTRRs enable
11	(all) MTRRs enable

Fixed Range MTRRs are a type of MTRR which operate on a fixed region of memory. There are 11 fixed-range registers of 64 bytes each, each is divided into eight 8-bit fields used to specify the memory type for the sub-ranges of the register. The IA32_MTRR_FIX64K_00000 MSR divides the 512-KiB range 7FFFF:00000 into eight 64-KiB sub-ranges. The IA32_MTRR_FIX16K_80000 and IA32_MTRR_FIX16K_A0000 MSRs divide the two 128-KiB ranges within the range BFFFF:80000 into sixteen 16-KiB sub-ranges (8 per register). The IA32_MTRR_FIX4K_C0000 through IA32_MTRR_FIX4K_F8000 MSRs divide the eight 32-KiB ranges within the range FFFFF:C0000 into sixty-four 4-KiB sub-ranges (8 per register).

Variable Range MTRRs are a type of MTRR which operate on a variable region of memory. The number (m) of variable range MTRRs supported is given in bits 7:0 of the IA32_MTRRCAP MSR. The entry in IA32_MTRR_PHYSBASEn defines the base address and memory type for the range, the entry in IA32_MTRR_PHYSMASKn contains a mask used to determine the range, where n is an integer in the range 0 to m-1.

Layout of IA32_MTRR_PHYSBASEn MSR:

Bits	Use
7:0	Memory type
12:MAXPHYADDR-1	Base address

Layout of IA32_MTRR_PHYSMASKn MSR:

Bits	Use
11	Valid (enables the MTRR entry)
12:MAXPHYADDR-1	Mask

An address is within the range of a variable MTRR when: `Address AND Mask = Base AND Mask`. Furthermore there are STRRs (System-Management Range Registers) used in System Management Mode.

[3, Chapter 13.11.2]

The MTRR memory type is determined as follows:

- If MTRRs are not enabled then all memory accesses are UC.
- Else if the physical address is within the first MiB and fixed MTRRs are enabled then it uses the appropriate fixed-range MTRR.
- Else if the address falls within a variable MTRR then it is used. If it falls within multiple variable MTRRs and they are not identical then, UC is used if one is UC, WT is used if they are all WT or WB otherwise it is undefined.
- Otherwise it uses the default.

[3, Chapter 13.11.4.1]

3.2.4.2 Page Attribute Table

The Page Attribute Table (PAT) provides additional functionality to set the memory type for page entries.
[3, Chapter 13.12]

Furthermore, PAT is supported on all 64-bit processors.
[3, Chapter 5.9.1]

Memory Types that can be encoded with PAT:

Encoding	Memory Type
00	UC
01	WC
02	Rsvd.
03	Rsvd.
04	WT
05	WP
06	WB
07	UC-
08-FF	Rsvd.

The IA32_PAT MSR stores eight PAT entries, each of which stores a memory type and can be selected by page entries.

[3, Chapter 13.12.2]

PAT Entries:

PAT	PCD	PWT	PAT Entry	Default Memory Type
0	0	0	PAT0	WB
0	0	1	PAT1	WT
0	1	0	PAT2	UC-
0	1	1	PAT3	UC
1	0	0	PAT4	WB
1	0	1	PAT5	WT
1	1	0	PAT6	UC-
1	1	1	PAT7	UC

Updating the PAT usually requires the OS to invalidate all affected TLB entries

[3, Chapter 13.12.3] [3, Chapter 13.12.4]

3.2.4.3 Effective Memory Type

The effective memory type is based on a combination of the MTRR memory type and the PAT (or PCD and PWT if PAT is unsupported) for the memory address. If Paging is not used, then the effective memory type is that of the MTRR. Furthermore if CR0.CD is set then caching is prevented.

[3, Chapter 13.5.2]

Effective Memory Type (PAT Unsupported):

Page Flags		**MTRR**				
PCD	**PWT**	**UC**	**WC**	**WT**	**WB**	**WP**
0	**0**	UC	WC	WT	WP	WB
0	**1**	UC	WC*	WT	WP*	WT
1	**0**	UC	WC*	UC	WC*	UC
1	**1**	UC	UC	UC	UC	UC

* Implementation defined, avoid these combinations.

[3, Chapter 13.5.2.1]

Effective Memory Type (PAT Supported):

		MTRR				
		UC	**WC**	**WT**	**WB**	**WP**
PAT	**UC**	UC1	UC2	UC2	UC2	UC2
	UC-	UC1	WC	UC2	UC2	WC
	WC	WC	WC	WC	WC	WC
	WT	UC1	UC2	WT	WT	WT
	WB	UC1	WC	WT	WB	WP
	WP	UC1	UC2	WP	WP	WP

UC1 - UC attribute comes from MTRRs so processors are not required to snoop their caches since the data could never have been cached. Preferred over UC2 for performance Reasons.
UC2 - UC attribute comes from PAT so processors are required to snoop their caches since the data may be cached due to page aliasing (which is not recommended).

[3, Chapter 13.5.2.2]

3.3 Memory addressing

Memory addresses are used to reference specific locations in memory (which can be accessed by dereferencing the address), (effective) memory addresses in assembly code have the syntax: `[Base + Index * Scale + Displacement]`. Furthermore a segment override can be added using the syntax: `Segment:[...]`.

x86 has different types of memory address:

1. Logical:
   Consists of a segment selector (16-bit) and an offset (same bit size as the code). Usually inferred from the type of memory access (eg uses CS for code, SS for stack) or from a segment override. Produces a linear address after segmentation.

2. Effective:
   Consists of a combination of a `Base` register, an `Index` register (optionally with an immediate `Scale` which is 1, 2, 4, or 8) and an immediate `Displacement`. Once the effective address is computed it is used as the offset in the logical address.

3. Linear:
   If paging is not used, the linear address is equal to the physical address. If paging is used, the corresponding physical address is found using the paging hierarchy.

4. Virtual:
   A valid linear address that does not correspond to an in-memory page, causing the kernel to generate the page via an interrupt (eg load it from disk).

5. Physical:
   The physical location of the memory in main memory.

3.4 Segmentation

Segmentation is a method of memory management that separates parts of a processes address space into different segments (such as code and data), it uses the 6 segment registers to keep track of the segments. It is used to convert a logical address (which consists of a segment register and an offset) into a linear address. Segmentation has different behaviours depending on what operating mode the processor is in, unlike paging it is always enabled.

[3, Chapter 3.1]

3.4.1 Real Mode

The value in the segment register is left shifted by 4 (multiplied by 16) then added to the offset to produce a 20-bit physical address allowing up to 1MiB of memory instead of the 16-bit limit of 64KiB [3, Chapter 22.1.1]. The kernel allocates memory for each running process and gives them each different segments, this allows the programs to be able to be loaded into any part of the address space and to not affect other program's memory. However there is no hardware enforcement of memory protection as any process can read and modify any memory location.

3.4.2 Protected Mode

The value in the segment register is a segment selector, it is used as an index into the LDT/GDT to select a segment descriptor. Each segment descriptor contains a base address, limit and flags. The base address is a linear address and points to the base of the segment, the limit is the size of the segment and the flags set the segments properties such as permissions, 32 or 16 bit code and granularity of the limit. There are many models which can be used to implement segmentation; some common models are listed below, other models exist and additional functionality can be added to the models such using the code segment to point to ROM.

[3, Chapter 3.1]

1. Basic Flat Model [3, Chapter 3.2.1]:
   Contains at least two segment descriptors, one for code and one for data (this is because the code descriptor has to be executable so is usually read-only). Both descriptors are mapped to the entire linear address space (even if the physical memory does not exist) so it has a base address of 0 and a limit of 4GiB. This essentially disables segmentation for both the application and system developer, and since the entire linear address space is mapped no #GP exceptions will occur. Usually implemented alongside paging.

2. Protected Flat Model [3, Chapter 3.2.2]:
   Similar to the Basic Flat Model but only included the range of addresses for which physical memory actually exists, so the limit is set to the capacity of physical memory. If a process accesses a non-existent memory location a #GP exception will occur.

3. Multi-Segment Model [3, Chapter 3.2.3]:
   Uses the full capabilities of segmentation to provide hardware enforced protection of memory. Each program is given its own segment descriptor table and multiple segments (which may be private or shared with other programs). Hardware checks can be used to enforce the program to access memory it owns and in a way the OS allows (such as not writing to code/read-only memory)

3.4.3 Long Mode

The behaviour of segmentation in long mode depends if it is running in 64-bit or compatibility mode. In compatibility mode it functions the same as it does in protected mode. In 64-bit mode, segmentation is generally disabled (but not completely), which creates a flat 64-bit linear address space. The base address of the CS, DS, ES and SS are treated as 0 (the effective address is equal to the linear address), and the processor does not perform limit checks (on all segment registers). The FS and GS registers are often used to reference thread-local or kernel data structures since their base address still applies, however they are stored in the IA32_FS_BASE and IA32_GS_BASE MSRs respectively.

[3, Chapter 3.2.4]

3.4.4 Segment Selectors

Segment selectors are used as a 16-bit identifier for a segment descriptor.

Segment selectors contain 3 parts:

1. Bits 15:3 - Index:
   The index into the descriptor table, selects one of the 8192 segment descriptors in the table. Its value is multiplied by 8 then added to the base address of the of the descriptor table.

2. Bit 2 - Table Indicator (TI) flag:
   Selects between the two descriptor tables: 0 selects the GDT to be used, 1 selects the LDT to be used.

3. Bits 1:0 - Requested Privilege Level (RPL):
   Specifies the privilege level of the selector.

If the index is 0, it implicitly selects a null segment. Additionally both the CS and SS registers have to be loaded with a present descriptor else a #GP exception is generated; furthermore if the SS is loaded with a non-writeable segment, a #GP is generated.

[3, Chapter 3.4.2]

3.4.5 Segment Descriptors

Segment descriptors are identifiers for segments of memory. They contain multiple parts which can be categorised into: (linear) base address of the segment, limit (size of segment - 1), and flags (controls the abilities of the segment).

Layout of segment descriptors:

Bits	**Size**	**Name**	**Use**
63:56	8	Base 31:24	1 byte of the base address
55:55	1	Granularity (G)	0 = limit in bytes, 1 = limit in 4-KiB
54:54	1	D/B	Default operation size, 0 = 16-bit, 1 = 32-bit
53:53	1	L	1 = 64-bit code segment (IA-32e mode only)
52:52	1	AVL	Available for use by system software
51:48	4	Limit 19:16	4 bits of the limit
47:47	1	Present (P)	0 = segment not present, 1 = segment present
46:45	2	DPL	Descriptor privilege level
44:44	1	S	Descriptor type, 0 = system, 1 = code/data
43:40	4	Type	Segment type
39:16	24	Base 23:00	3 byte of the base address
15:00	16	Limit 15:00	2 bytes of the base address

[3, Chapter 3.4.5]

Segment types:

	Type					
S	**3**	**2**	**1**	**0**	**32-bit mode**	**IA-32e mode**
0	0	0	0	0	Reserved	Reserved
0	0	0	0	1	16-bit TSS (Available)	Reserved
0	0	0	1	0	LDT	LDT
0	0	0	1	1	16-bit TSS (Busy)	Reserved
0	0	1	0	0	16-bit Call Gate	Reserved
0	0	1	0	1	Task Gate	Reserved
0	0	1	1	0	16-bit Interrupt Gate	Reserved
0	0	1	1	1	16-bit Trap Gate	Reserved
0	1	0	0	0	Reserved	Reserved
0	1	0	0	1	32-bit TSS (Available)	64-bit TSS (Available)
0	1	0	1	0	Reserved	Reserved
0	1	0	1	1	32-bit TSS (Busy)	64-bit TSS (Busy)
0	1	1	0	0	32-bit Call Gate	64-bit Call Gate
0	1	1	0	1	Reserved	Reserved
0	1	1	1	0	32-bit Interrupt Gate	64-bit Interrupt Gate
0	1	1	1	1	32-bit Trap Gate	64-bit Trap Gate
-	**-**	**E**	**W**	**A**	**Type**	**Description**
1	0	0	0	0	Data	Read-Only
1	0	0	0	1	Data	Read-Only, accessed
1	0	0	1	0	Data	Read-Write
1	0	0	1	1	Data	Read-Write, accessed
1	0	1	0	0	Data	Read-Only, expand-down
1	0	1	0	1	Data	Read-Only, expand-down, accessed
1	0	1	1	0	Data	Read-Write, expand-down
1	0	1	1	1	Data	Read-Write, expand-down, accessed
-	**-**	**C**	**R**	**A**	**Type**	**Description**
1	1	0	0	0	Code	Execute-Only
1	1	0	0	1	Code	Execute-Only, accessed
1	1	0	1	0	Code	Execute-Read
1	1	0	1	1	Code	Execute-Read, accessed
1	1	1	0	0	Code	Execute-Only, conforming
1	1	1	0	1	Code	Execute-Only, conforming, accessed
1	1	1	1	0	Code	Execute-Read, conforming
1	1	1	1	1	Code	Execute-Read, conforming, accessed

Description of bits used by the segment types:

1. E = Expansion-direction:
   If 0, the segment is expand-up; this means that the range of valid offsets is [0, limit]. Therefore the range of linear addresses of the segment is [base, base + limit], increasing the limit increases the maximum address (useful for data segments). If 1, the segment is expand-down; this means that the range of valid offsets is (limit, 0xFFFFF] if B = 0 or (limit, 0xFFFFFFFF] if B = 1. Therefore the range of linear addresses of the segment is (base + limit, base + (0xFFFFF or 0xFFFFFFFF)], decreases the limit decreases the minimum address (useful for stack segments).

2. W = Write-enabled:
   Allows the segment to be written; the stack segment must be writeable, loading the SS register with a read only segment causes a #GP exception.

3. A = Accessed:
   The processor sets it to 1 if it accessed the segment. The kernel can clear its value and it is often useful for debugging. If the segment descriptor is placed in ROM, the processor may enter an infinite loop attempting to set the Accessed bit. To prevent this, the segment descriptors in ROM should have their Accessed bit set.

4. C = Conforming:
   A transfer of execution into a more privileged conforming segment will continue at the current privilege level. Transferring execution into a more privileged non-conforming segment or less privileged segment will result in a #GP exception if a call or task gate is not used. All data segments are non-conforming so cannot be accessed by less privileged processes, however unlike code segments higher privileged processes can access lower privileged data segments without the use of a gate.

5. R = Read-enabled:
   Allows the code segment to be read and is useful if constants or static data are placed within the code segment. The code segments can be read either by using a CS override or by loading the segment into a data-segment register (DS, ES, GS, FS).

[3, Chapter 3.4.5] [3, Chapter 3.4.5.1] [3, Chapter 3.5]

Furthermore, when a segment register is loaded with a segment selector, the processor stores the corresponding descriptor in the hidden part of the segment register (the descriptor cache), if the descriptor is changed it is the responsibility of the software to reload the segment selector [3, Chapter 3.4.3].

Additionally in IA-32e mode, system descriptors are expended to 16 bytes (occupying the space of two entries) [3, Chapter 3.52].

3.4.6 Segment Descriptor Tables

The Global Descriptor Table (GDT) contains segment descriptors, its base address (linear) and limit (number of bytes in the GDT - 1) are stored in the GDTR (GDT Register). The `LGDT` instruction stores the base address and limit from a memory location into the GDTR. The 0th descriptor of the GDT is often used to store the address and limit since it will never be loaded as a descriptor since the selector would be null. There is usually only a single GDT used by all processes.

The Local Descriptor Table (LDT) also contains segment descriptors; however unlike the GDT there are usually many LDTs (in a non-flat segmentation model), this is so that each process can have its own set of segments. Unlike the GDTR, the LDTR (LDT Register) stores the segment selector for the LDT table segment stored in the GDT. The `LLDT` instruction stores the segment selector of the new LDT in the LDTR. Furthermore in 32-bit task switches the LDTR is automatically loaded with the segment selector and descriptor for the LDT of the new task.

[3, Chapter 3.5.1]

3.5 Paging

Paging is a method of memory management that separates the linear address space from the physical address space. It occurs after segmentation but unlike segmentation it does not need to be enabled, if it is disabled linear addresses are the same as physical addresses. The OS splits up the physical address space into pages, each process has a paging hierarchy which the OS manages, it consists of many paging structures. When a process accesses memory, the processor translates the linear address into its corresponding physical address using the paging hierarchy (by traversing the paging hierarchy), this is called a hardware-assisted page walk. There are four different paging modes.

3.5.1 Enabling Paging

Before paging is enabled, CR3 should contain the physical address of the first paging structure, and protection (CR0.PE) is enabled. Furthermore, once paging is enabled a different paging mode can be used but paging should be disabled first before changing flags since exceptions may occur by changing the flags whilst paging is enabled. Paging is enabled by setting CR0.PG.

[3, Chapter 5.1.2]

The following table shows which combination of flags is used to enable which paging mode:

Mode	CR4.PAE	IA32_EFER.LME	CR4.LA57
32-bit	0	0	-
PAE	1	0	-
4-level	1	1	0
5-level	1	1	1

Since IA32_EFER.LME enables long mode (IA-32e), 32-bit and PAE can only be used in legacy protected mode and 4-level and 5-level can only be used in long mode. Furthermore, in compatibility mode (sub mode of IA-32e), bits 63-32 of the linear address are treated as 0.

[3, Chapter 5.1.1]

3.5.2 Paging Structures

All four paging modes translate linear addresses using hierarchical paging structures. Each structure is 4096 bytes (4-KiB) in size and comprises several entries, in 32-bit paging each entry is 32 bits (4 bytes) hence there are 1024 entries in each structure, in the other paging modes each entry is 64 bits (8 bytes) hence there are 512 entries in each structure. However in PAE paging, there is a structure 32 bytes in size containing 4 64-bit entries.

The processor uses the upper portion of a linear address to identify a series of paging-structure entries. The last of these entries identifies the physical address of the region to which the linear address translates (called the page frame). The lower portion of the linear address (called the page offset) identifies the specific address within that region to which the linear address translates. Each paging structure entry contains a physical address which is either the address of another paging structure or a page frame. In the first case it is said to reference the other paging structure; in the second the entry is said to map a page. CR3 contains the physical address of the first paging structure (unless HLAT paging is used).

Linear addresses are translated into physical addresses using an iterative procedure: a portion of the linear address (initially the uppermost bits) is used to select an entry in a paging structure (initially the one referenced in CR3), if the entry references another structure then the process continues using another portion of the linear address (immediately below the portion just used), if the entry maps a page then the remaining lower portion of the linear address is used as a page offset.

A paging-structure entry is marked as not present if its P flag (bit 0) is clear, if it is encountered during translation it causes a page-fault exception; if the exception handler returns, the instruction that generated the exception is restarted. Usually an address is mapped to a 4 KiB page, if it is not the final paging structure then bit 7 (PS - Page Size) is checked; if PS = 0 then it references a page structure, else if PS = 1 then the entry maps a larger page (4MiB for 32-bit paging if CR4.PSE = 1, 2MiB for other paging modes regardless of CR4.PSE; 4-level and 5-level paging may also support 1GiB pages),

[3, Chapter 5.2]

The following table lists paging structures along with: the structure that stores their physical address (CR3/previous structure), the bits of the linear address that is used to select the entry within them, and the size of the page they map to (if they map to a page).

Paging Structure	**Entry Name**	**Paging Mode**	**Physical Address of Structure**	**Bits Selecting Entry**	**Page Mapping**
PML5 table	PML5E	32-bit, PAE, 4-level	N/A		
		5-level	CR3	56:48	N/A (PS must be 0)
PML4 table	PML4E	32-bit, PAE	N/A		
		4-level	CR3	47:39	N/A (PS must be 0)
		5-level	PML5E		
Page-directory-pointer table	PDPTE	32-bit	N/A		
		PAE	CR3	31:30	N/A (PS must be 0)
		4-level, 5-level	PML4E	38:30	1-GiB page if PS=1
Page directory	PDE	32-bit	CR3	31:22	4-MiB page if PS=1
		PAE, 4-level, 5-level	PDPTE	29:21	2-MiB page if PS=1
Page table	PTE	32-bit	PDE	21:12	4-KiB page
		PAE, 4-level, 5-level		20:12	

[3, Chapter 5.2]

The following tables describe the layout of the paging structures (M = MAXPHYADDR, * = Pointer to next paging structure or page frame).

32-bit Paging:

Bits	**CR3**	**PDE: 4MB**	**PDE: PTE**	**PTE: 4KB**	**Not Present**
31:22	PD*	31:22 Page*	PT*	Page*	Ign.
21:17		Rsvd.			
16:13		39:32 Page*			
12		PAT			
11	Ign.	Ign.	Ign.	Ign.	
10					
9					
8		G		G	
7		1	0	PAT	
6		D	Ign.	D	
5		A	A	A	
4	PCD	PCD	PCD	PCD	
3	PWT	PWT	PWT	PWT	
2	Ign.	U/S	U/S	U/S	
1		R/W	R/W	R/W	
0		1	1	1	0

[3, Chapter 5.3]

PAE Paging:

<table>
<tr><th>Bits</th><th>CR3</th><th>PDPTE</th><th>PDE: 2MB</th><th>PDE: PTE</th><th>PTE: 4KB</th><th>Not Present</th></tr>
<tr><td>63</td><td rowspan="3">Ign.</td><td rowspan="2">Rsvd.</td><td>XD</td><td>XD</td><td>XD</td><td rowspan="19">Ign.</td></tr>
<tr><td>62:M</td><td>Rsvd.</td><td>Rsvd.</td><td>Rsvd.</td></tr>
<tr><td>M-1:32</td><td rowspan="4">PDE*</td><td rowspan="2">Page*</td><td rowspan="4">PT*</td><td rowspan="4">Page*</td></tr>
<tr><td>31:21</td><td rowspan="9">PDPT*</td></tr>
<tr><td>20:13</td><td>Rsvd.</td></tr>
<tr><td>12</td><td>PAT</td></tr>
<tr><td>11</td><td rowspan="3">Ign.</td><td rowspan="3">Ign.</td><td rowspan="3">Ign.</td><td rowspan="3">Ign.</td></tr>
<tr><td>10</td></tr>
<tr><td>9</td></tr>
<tr><td>8</td><td rowspan="4">Rsvd.</td><td>G</td><td>G</td></tr>
<tr><td>7</td><td>1</td><td>0</td><td>PAT</td></tr>
<tr><td>6</td><td>D</td><td>Ign.</td><td>D</td></tr>
<tr><td>5</td><td>A</td><td>A</td><td>A</td></tr>
<tr><td>4</td><td rowspan="5">Ign.</td><td>PCD</td><td>PCD</td><td>PCD</td><td>PCD</td></tr>
<tr><td>3</td><td>PWT</td><td>PWT</td><td>PWT</td><td>PWT</td></tr>
<tr><td>2</td><td rowspan="2">Rsvd.</td><td>U/S</td><td>U/S</td><td>U/S</td></tr>
<tr><td>1</td><td>R/W</td><td>R/W</td><td>R/W</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr>
</table>

[3, Chapter 5.4]

4/5 Level paging:

<table>
<tr><th>Bits</th><th>CR3</th><th>PML5E: PLM4E</th><th>PLM4E: PDPTE</th><th>PDPTE: 1GB</th><th>PDPTE: PDE</th><th>PDE: 2MB</th><th>PDE: PTE</th><th>PTE: 4KB</th><th>Not Present</th></tr>
<tr><td>63</td><td rowspan="3">Rsvd.</td><td>XD</td><td>XD</td><td>XD</td><td>XD</td><td>XD</td><td>XD</td><td>XD</td><td rowspan="22">Ign.</td></tr>
<tr><td>62:59</td><td rowspan="2">Ign.</td><td rowspan="2">Ign.</td><td>Prot</td><td rowspan="2">Ign.</td><td>Prot</td><td rowspan="2">Ign.</td><td>Prot</td></tr>
<tr><td>58:52</td><td>Ign.</td><td>Ign.</td><td>Ign.</td></tr>
<tr><td>51:M</td><td>Rsvd.</td><td>Rsvd.</td><td>Rsvd.</td><td>Rsvd.</td><td>Rsvd.</td><td>Rsvd.</td><td>Rsvd.</td></tr>
<tr><td>M-1:30</td><td rowspan="4">PML4*
/
PML5*</td><td rowspan="4">PML4*</td><td rowspan="4">PDPT*</td><td>Page*</td><td rowspan="4">PD*</td><td rowspan="2">Page*</td><td rowspan="4">PT*</td><td rowspan="4">Page*</td></tr>
<tr><td>29:21</td><td rowspan="2">Rsvd.</td></tr>
<tr><td>20:13</td><td>Rsvd.</td></tr>
<tr><td>12</td><td>PAT</td><td>PAT</td></tr>
<tr><td>11</td><td rowspan="7">Ign.</td><td>R</td><td>R</td><td>R</td><td>R</td><td>R</td><td>R</td><td>R</td></tr>
<tr><td>10</td><td rowspan="3">Ign.</td><td rowspan="3">Ign.</td><td rowspan="2">Ign.</td><td rowspan="3">Ign.</td><td rowspan="2">Ign.</td><td rowspan="3">Ign.</td><td rowspan="2">Ign.</td></tr>
<tr><td>9</td></tr>
<tr><td>8</td><td>G</td><td>G</td><td>G</td></tr>
<tr><td>7</td><td>Rsvd.</td><td>Rsvd.</td><td>1</td><td>0</td><td>1</td><td>0</td><td>PAT</td></tr>
<tr><td>6</td><td>Ign.</td><td>Ign.</td><td>D</td><td>Ign.</td><td>D</td><td>Ign.</td><td>D</td></tr>
<tr><td>5</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td></tr>
<tr><td>4</td><td>PCD</td><td>PCD</td><td>PCD</td><td>PCD</td><td>PCD</td><td>PCD</td><td>PCD</td><td>PCD</td></tr>
<tr><td>3</td><td>PWT</td><td>PWT</td><td>PWT</td><td>PWT</td><td>PWT</td><td>PWT</td><td>PWT</td><td>PWT</td></tr>
<tr><td>2</td><td rowspan="3">Ign.</td><td>U/S</td><td>U/S</td><td>U/S</td><td>U/S</td><td>U/S</td><td>U/S</td><td>U/S</td></tr>
<tr><td>1</td><td>R/W</td><td>R/W</td><td>R/W</td><td>R/W</td><td>R/W</td><td>R/W</td><td>R/W</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr>
</table>

[3, Chapter 5.5]

Description of many of the bits/labels used in the page entries:

Label	Name	Description
P (Bit 0)	Present	If 1 it maps to the next structure else it raises a #PF
R/W	Read/Write	If 0, writes are not allowed
U/S	User/Supervisor	If 0, user-mode accesses are not allowed
PWT	Page-level Write-Through	Indirectly determines the memory type used to access the page
PCD	Page-level Cache Disable	Indirectly determines the memory type used to access the page
A	Accessed	Indicates whether the software has accessed the page
D	Dirty	Indicates whether the software has written to the page
PAT	Page Attribute Table	Indirectly determines the memory type used to access the page
G	Global	If 1 and CR4.PGE = 1, the translation is global
R	Restart	If 1 and HLAT paging, restart using ordinary paging
Prot	Protection key	If CR4.PKE = 1 or CR4.PKS = 1, the page's access rights, else ignored
XD	Execute Disable	If IA32_EFER.NXE = 1, Disabled instruction fetches
Ign.	Ignored	Bits are not used
Rsvd.	Reserved	Bits must be 0

[3, Chapter 5.3] [3, Chapter 5.4] [3, Chapter 5.5]

3.5.3 Translation Lookaside Buffer

The Translation Lookaside Buffer (TLB) is a specialised cache which stores linear address translations, they contain entries that map page numbers to page frames. [3, Chapter 5.10.2]

Each entry in a TLB is an individual translation; each entry is referenced by a page number and stores: physical address of the page frame, access rights, attributes (dirty flag, memory type). Other information may also be stored and special-use TLBs may not store all this information (eg instruction TLB would not store the dirty flag). TLB entries are associated with the current PCID when they are created. [3, Chapter 5.10.2.2]

The processor may cache translations that were not used (eg due to speculative execution), furthermore some processors may cache a large page (one greater than 4-KiB) as multiple 4-KiB TLB entries. [3, Chapter 5.10.2.3]

In addition to TLBs there are paging structure caches. There is a cache specific to each paging structure, each entry in these caches contains information from the paging structure (eg its physical address and flags) and the bits from the linear address used to identify the entry. [3, Chapter 5.10.3]

To ensure the paging caches contain up to date information, the OS must invalidate any cached entries when they are updated. [3, Chapter 5.10.4]

In multiprocessor systems the OS may be required to invalidate the cache in other processors, this requires the OS to perform a TLB shootdown; a shootdown can be achieved by using an interprocessor interrupt (IPI) to interrupt all processors and use the interrupt handler to invalidate the cache. [3, Chapter 5.10.5]

3.6 Virtual Memory

Virtual memory is a method of extending main memory by temporarily moving sections of it into secondary storage then moving them back when needed. While it is possible to implement virtual memory through segmentation (by moving entire segments in and out of secondary storage), it is most commonly implemented through paging.

If the OS runs out of memory, it may move several (rarely used) pages into secondary storage (often into a swap file or swap partition), and clearing the present flag for its corresponding page entry. If a process attempts to access one of these pages, the page walk will encounter a non-present entry and a #PF exception will be generated. The exception handler will be called and will move the page back into main memory, updating the page entry to the new physical address and setting the present flag. When the exception handler returns, the instruction that generated the exception is restarted.

[3, Chapter 5.12]

4 Task Management

Task management can be used to switch the currently executing task by dispatching a new task and suspending the current task. A task is a unit of work that the processor can dispatch, execute, and suspend. It can be used to execute a process, task, process, interrupt handler, etc. All processor execution takes place from within a task, even simple systems must define at least one task.

[3, Chapter 9.1]

A task is made up of two parts: a task execution space and a Task-State Segment (TSS). The execution space consists of a code, stack and at least one data segment, if privilege levels are used then it also provides a separate stack for each privilege level. The TSS specifies the segments that make up the task execution space and provides a storage space for the task state, in multitasking systems it also provides a mechanism for linking tasks. A task is identified by the segment selector for its TSS, when a task is loaded its TSS segment descriptor is loaded into the task register. If paging is used, the base address for the paging hierarchy is loaded into CR3.

[3, Chapter 9.1.1]

The state of the currently executing task is defined by: segment registers, general purpose registers, EFLAGS, EIP, CR3, Task register, LDT segment selector, I/O map base address and I/O map, stack pointers to privilege 0, 1, 2 stacks, a link to previously executed task, and state of the shadow stack pointer. All of these are stored in TSS except task register.

[3, Chapter 9.1.2]

The Task Register (TR) holds the segment selector and segment descriptor for the TSS of the current task. The `LTR` and `STR` instructions load and store the task register with a TSS descriptor. [3, Chapter 9.2.4]

When in protected mode there must be at least one TSS and one TSS descriptor, and the segment selector for the TSS must be loaded into the task register. [3, Chapter 9.2]

Software can dispatch a task by: an explicit call to a task, explicit jump to a task, implicit call to an interrupt-handler task, or an IRET when EFLAGS.NT is set. A task switch occurs between the currently running and the dispatched task, the current execution context of the current task is saved in its TSS and its execution is suspended, the context for the dispatched task is loaded into the processor and execution resumes with the new task. If the current task called the dispatched task, its TSS selector is stored in the TSS of the dispatched task to provide a link back to the previous task.

Interrupts can be handled using a task switch, upon returning it automatically switches back to the interrupted task. During a switch the processor can also switch the LDT and CR3 allowing each task to have a different LDT and page hierarchy. Tasks cannot jump to themselves, and the use of tasks and task switching is optional.

[3, Chapter 9.1.3]

EFLAGS.NT indicates if the current task is nested within the execution of another, it then uses the previous task link field to return to the previous task. Using `CALL` nests a task while `JMP` does not.

[3, Chapter 9.4]

4.1 Segment Descriptors

There are two different segment descriptors used for handling the TSS: TSS descriptor and task-gate descriptor.
The TSS descriptor can only be stores in the GDT and has the same format as segment descriptors, its D/B and L flags are set to 0. Its limit must be large enough to contain the entire TSS, larger limits are used for I/O maps. Only processes whose CPL is equal to or less than the DPL of the TSS can dispatch it with a call or jump.

[3, Chapter 9.2.2]

A task-gate descriptor provides an indirect protected reference to a task. The RPL of its segment selector is not used, when a program makes a call/jump to a task through a task gate, the CPL and RPL of the selector must be less than or equal to the gate's DPL.

Layout of the task-gate descriptor:

Bits	Description
63:48	Rsvd.
47	0 = Not Present, 1 = Present
46:45	Descriptor Privilege Level (DPL)
44:40	00101
39:32	Rsvd.
31:16	TSS Segment Selector
15:0	Rsvd.

[3, Chapter 9.2.5]

4.2 16-bit TSS

The 16-bit TSS cannot be used to implement a virtual-8086 task, furthermore the I/O map is not supported.

Format of the 16-bit TSS:

Bytes	Description
1:0	Previous Task Link
3:2	SP0
5:4	SS0
7:6	SP1
9:8	SS1
11:10	SP2
13:12	SS2
15:14	IP
17:16	FLAG
19:18	AX
21:20	CX
23:22	DX
25:24	BX
27:26	SP
29:28	BP
31:30	SI
33:32	DI
35:34	ES
37:36	CS
39:38	SS
41:40	DS
43:42	LDT Selector

[3, Chapter 9.6]

4.3 32-bit TSS

Layout of the 32-bit TSS:

Bytes	Description
1:0	Previous Task Link
3:2	Rsvd.
7:4	ESP0
9:8	SS0
11:10	Rsvd.
15:12	ESP1
17:16	SS1
19:18	Rsvd.
23:20	ESP2
25:24	SS2
27:26	Rsvd.
31:28	CR3
35:32	EIP
39:36	EFLAGS
43:40	EAX
47:44	ECX
51:48	EDX
55:52	EBX
59:56	ESP
63:60	EBP
67:64	ESI
71:68	EDI
73:72	ES
75:74	Rsvd.
77:76	CS
79:78	Rsvd.
81:80	SS
83:82	Rsvd.
85:84	DS
87:86	Rsvd.
89:88	FS
91:90	Rsvd.
93:92	GS
95:94	Rsvd.
97:96	LDT Segment Selector
99:98	Rsvd.
101:100	Rsvd.*
103:102	I/O Map Base Address
107:104	SSP

* If bit 0 is set then a debug exception is raised when a task switch to this task occurs.

The TSS contains both dynamic and static fields, dynamic fields are changed by the processor when the task saves state, static fields are not changed. The dynamic fields are: general purpose registers, segment selectors, EFLAGS, EIP, and Previous Task Link (previous TSS segment selector), all other fields are static. The SS and ESP fields are logical addresses used for privilege levels 0, 1, and 2. SSP contains the shadow stack pointer.

[3, Chapter 9.2.1]

4.4 64-bit mode

Task switching is not supported in 64-bit mode and must be performed by software. The 64-bit TSS contains stack pointers for privilege levels 0-2, interrupt stack table pointers and the I/O map base address.

Format of the 64-bit TSS:

Bytes	Description
3:0	Rsvd.
11:4	RSP0
19:12	RSP1
27:20	RSP2
35:28	Rsvd.
43:36	IST1
51:44	IST2
59:52	IST3
67:60	IST4
75:68	IST5
83:76	IST6
91:84	IST7
101:92	Rsvd.
103:102	I/O Map Base Address

[3, Chapter 9.7]

4.5 Call Gates

Call gates facilitate controlled transfers of program control between different privilege levels. Typically only used in operating systems that use privilege-level protection, also useful for transferring control between 16-bit and 32-bit code segments.

Format of a 16/32-bit call-gate descriptor:

Bits	Name	Use
63:48	Offset 31:16	Entry point in segment
47:47	Present (P)	0 = Not present, 1 = Present
46:45	DPL	Descriptor privilege level
44:36	01100000	Must be 01100000
35:32	Param. Count	Parameter Count
31:16	Segment Selector	Code Segment Selector
15:0	Offset 15:0	Entry point in segment

If a stack switch occurs, the parameter count specifies the number of optional parameters to be copied between stacks. The size of values to be pushed onto the target stack is equal to the size of the gate (determined by the size of the code segment). The DPL is the privilege level required to access the procedure through the gate.

[3, Chapter 6.8.3]

Format of a 64-bit call-gate:

Bits	Name	Use
127:96	Rsvd.	Must be 0
95:48	Offset 63:16	Entry point in segment
47:47	Present (P)	0 = Not present, 1 = Present
46:45	DPL	Descriptor privilege level
44:32	0110000000000	Must be 0110000000000
31:16	Segment Selector	Code Segment Selector
15:0	Offset 15:0	Entry point in segment

Unlike 16/32-bit gates, parameter copying is not performed.

[3, Chapter 6.8.3.1]

5 Instructions

This section contains many of the x86 instructions and their use, it is not an exhaustive list and is a simplification of how many instructions work (by not including many of the potential exceptions which may be raised, not including how the instructions are encoded in machine code etc, or the flags set/cleared), however it should be sufficient enough to create simple programs. The format of the instruction is `instruction op1, op2, op3` eg `'MOV RAX, 10'`.

Each instruction has different limitations to what type an operand can be, these types are:

Symbol	Name	Description
r	General purpose register	A register such as eax
m	Memory address	A memory address such as `[eax + 4*ebx + 4]`
i	Immediate	A constant value such as 80h or a pointer to a label
s	Segment Register	A register such as ES
d	Debug Register	A register such as DR0
c	Control Register	A register such as CR3

The types of an operand or identical instructions with different names will be separated by `(|)`, the size in bits may also be appended to the type and (') may be used to exclude a single size (eg `i8|i16` means an 8 or 16 bit immediate value and `r'8` means a 16, 32 or 64 bit register). Some instructions have more than one row of operands, each row shows a different combination of operand types. Most instructions should only operate on a single word size (eg 64-bit), if an instruction is used with different sized operands then it may cause an assembler error, cause an invalid opcode exception, truncate the source or bit extend it (either arithmetically or logically). Furthermore many of these instructions set/clear flags, for example the OF (overflow flag) is set on signed overflow, CF (carry flag) is set on unsigned overflow and ZF (zero flag) is set when the result is zero. Some instructions can be prefixed with LOCK, this makes the instruction atomic and will give the thread exclusive access to the cache line it is accessing; LOCK can only be used when the destination is a memory location and the instruction supports it else it will generate a #UD exception.

Instruction	OP 1	OP 2	OP 3
Memory			
`MOV` - Move [2, Page 4-35] Copies the value from OP2 into OP1	`r`	`r|m|i`	-
	`m`	`r|i'64`	-
	`d`	`r(32|64)`	-
	`c'CR8`	`r32`	-
	`c`	`r64`	-
	`s`	`(r|m)16`	-
	`(r|m)16`	`s`	-

Instruction	OP 1	OP 2	OP 3
`LEA` - Load Effective Address [2, Page 3-603] Stores the address from OP2 into OP1 but unlike MOV it does not dereference the address	r'8	m	-
`[LOCK] CMPXCHG` - Compare and Exchange [2, Page 3-212] Compares `AL|AX|EAX|RAX` with OP1 (dst), if equal OP2 (src) is written into OP1 and ZF is set else OP1 is written into the accumulator and ZF is cleared	r|m	r	-
`[LOCK] XCHG` - Exchange [2, Page 6-32] Swaps (exchanges) the values held in OP1 and OP2	r	r|m	-
	r|m	r	-
`IN` - Input From Port [2, Page 3-512] Copies the value from the I/O port specified by OP2 into OP1	al|ax|eax	dx|i8	-
`OUT` - Output To Port [2, Page 4-178] Copies the value from OP2 into the I/O port specified by OP1	dx|i8	al|ax|eax	-
Control			
`UD` - Undefined Instruction [2, Page 4-730] Causes an invalid opcode exception	-	-	-
`NOP` - No Operation [2, Page 4-167] Performs no operation, the first encoding is a single byte NOP and is an alias for XCHG (E)AX, (E)AX, and the second encoding is a multibyte NOP and may not be supported on all processors	-	-	-
	r|m	-	-
`CMP` - Compare Two Operands [2, Page 3-179] Compares OP1 with OP2, it does this by subtracting OP2 from OP1, setting the relevant flag and discarding the result	r	r|m	-
	r|m	r	-
	r|m	i'64	-
`TEST` - Logical Compare [2, Page 4-714] Performs a bitwise logical AND on OP1 and OP2, sets the relevant flags and discards the result	r|m	r	-
	r|m	i'64	-
`Jcc` - Jump If Condition Is Met [2, Page 3-555] Jumps if the condition is met	i'64	-	-
`JMP` - Jump [2, Page 3-560] Jumps to the location in code referenced by OP1. The first encodings are relative signed offsets to the current location in code (i8 is a short jump (type of near jump)), the rest are near jumps (jumps within the current code segment). The second encoding are near jumps to absolute addresses. The third encoding is an immediate of a 16-bit segment selector and an absolute offset into it and performs a far jump into another code segment. The fourth encoding is a memory location that points to a 16-bit segment selector followed by an absolute offset and performs a far jump	i'64	-	-
	(r|m)'8	-	-
	i16:(16|32)	-	-
	m16:('8)	-	-
Procedures			
`CALL` - Call Procedure [2, Page 3-139] Similar to `JUMP`. Additionally, in a near call the current instruction pointer is pushed to the stack. In a far call, both the CS and instruction pointer are pushed to the stack	i(16|32)	-	-
	(r|m)'8	-	-
	i16:(16|32)	-	-
	m16:('8)	-	-
`RET` - Return From Procedure [2, Page 4-567] Returns control to the return address located at the top of the stack (usually placed there by a CALL). OP1 specifies the number of bytes to be released from the stack after the return address is popped (default is 0)	-	-	-
	i16	-	-

Instruction	OP 1	OP 2	OP 3
`IRET|IRETD|IRETQ` - Interrupt Return [2, Page 3-546] Returns control to the interrupted program from the interrupt handler. It pops off the data pushed to the stack during the interrupt such as EFLAGS, Instruction Pointer and CS	-	-	-
`INT` - Call to Interrupt Procedure [2, Page 3-524] Generates a call to the interrupt handler specified by OP1	i8	-	-
Syscalls			
These instructions are used for syscalls (both calling and returning), SYSCALL and SYSRET are better supported on 64-bit CPUs and operating systems. All these instructions set the CS and SS segment selector registers to values defined in MSRs, and their descriptor caches to fixed values with base 0, limit 0xFFFFF, and flags as defined below (in 3 hexadecimal nibbles where the granularity flag is the most significant bit and the type is the last nibble). It is the responsibility of the OS to ensure that the fixed values of the descriptors match the descriptors referenced by the selectors			
`SYSCALL` - Fast System Call [2, Page 4-696] Invokes the OS system-call handler at privilege level 0, it does so by loading RIP from IA32_LSTAR MSR and saves the address of the instruction following SYSCALL into RCX. Furthermore RFLAGS is stored into R11, RFLAGS is then masked with NOT(IA32_FMASK MSR) (clears every bit in RFLAGS which is set in the MSR). The CS selector is loaded with IA32_STAR[47:32] AND 0xFFFC, and SS is loaded with IA32_STAR[47:32] + 8. The flags loaded into the CS and SS segment descriptors are 0xA9B and 0xC93 respectively	-	-	-
`SYSRET` - Return from Fast System Call [2, Page 4-705] Returns from an OS system-call handler to user code at privilege level 3, it is a companion instruction to SYSCALL. It loads RIP from RCX and RFLAGS from (R11 AND 0x3C7FD7H) OR 2. If the operand size is 64-bit it remains in 64-bit mode else it enters compatibility mode and only the low 32 bits of the registers are loaded. The CS selector is loaded with (IA32_STAR[63:48] + 16) OR 3 if the operand size is 64-bit else it is loaded with IA32_STAR[63:48] OR 3, and SS is loaded with (IA32_STAR[63:48] + 8) OR 3. The flags loaded into the CS segment descriptor are 0xAFB if the operand size is 64-bit and 0xCFB if not and 0xCF3 for the SS segment descriptor	-	-	-
`SYSENTER` - Fast System Call [2, Page 4-699] Executes a fast call to a level 0 system procedure or routine. In IA-32e mode, it transitions the processor to 64-bit mode, otherwise it remains in protected mode.It loads RIP (or EIP) from 63:0 (or 31:0) of IA32_SYSENTER_EIP, and RSP (or ESP) from 63:0 (or 31:0) of IA32_SYSENTER_ESP. The CS selector is loaded with IA32_SYSENTER_CS[15:0] AND 0xFFFC, and SS is loaded with the value stored in CS + 8. The flags loaded into the CS segment descriptor are 0xA9B in IA-32e mode and 0xC9B if not and 0xC93 for the SS segment descriptor. Since no return information is saved, it is the responsibility of the caller and the OS to manage the return information	-	-	-

Instruction	OP 1	OP 2	OP 3
`SYSEXIT` - Fast Return from Fast System Call [2, Page 4-702] Returns to privilege level 3 user code, it is a companion instruction to SYSENTER. If the operand size is 64-bit it remains in 64-bit mode, else it enters compatibility mode or remains in protected mode. RDX (or EDX) is loaded into RIP (or EIP) if the return is not to 64-bit mode then only bits 31:0 are loaded, and RCX (or ECX) is loaded into RSP (or ESP) if the return is not to 64-bit mode then only bits 31:0 are loaded. The CS selector is loaded with (IA32_SYSENTER_CS[15:0] + 32) OR 3 if the operand size is 64-bit else it is loaded with (IA32_SYSENTER_CS[15:0] + 16) OR 3, and SS is loaded with the value stored in CS + 8. The flags loaded into the CS segment descriptor are 0xAFB if the operand size is 64-bit and 0xCFB if not and 0xCF3 for the SS segment descriptor	-	-	-
Stack			
`PUSH` - Push data onto the stack [2, Page 4-520] Decrements the Stack Pointer by the operand size, then moves OP1 to the memory location pointed to by the it. It is recommended to only push values the same size as the pointer size to prevent a misaligned stack. Values less than the operand size are sign-extended to the operand size however segment registers are zero-extended. The operand size can be overridden by prefixes however its based on the current stack segment D flag. In long mode, 32-bit registers and the CS, DS, ES and SS registers cannot be encoded	`(r|m)'8`	-	-
	`i'64`	-	-
	`s`	-	-
`POP` - Pop data off the stack [2, Page 4-396] Moves the value pointed to by the Stack Pointer to OP1 then increments it by the operand size. Uses the same rules as PUSH	`(r|m)'8`	-	-
	`s`	-	-
Arithmetic			
`[LOCK] ADD` - Add [2, Page 3-32] Adds OP2 to OP1. If OP2 is an `immediate` value and its word size is less than OP1 then it is sign-extended	`r`	`r|m`	-
	`r|m`	`r`	-
	`r|m`	`i'64`	-
`[LOCK] INC` - Increment [2, Page 3-514] Increments the value in OP1 by 1	`r|m`	-	-
`[LOCK] ADC` - Add With Carry [2, Page 3-27] Adds (OP2 + CF) to OP1. If OP2 is an `immediate` value and its word size is less than OP1 then it is sign-extended	`r`	`r|m`	-
	`r|m`	`r`	-
	`r|m`	`i'64`	-
`[LOCK] SUB` - Subtract [2, Page 4-682] Subtracts OP2 from OP1. If OP2 is an `immediate` value and its word size is less than OP1 then it is sign-extended	`r`	`r|m`	-
	`r|m`	`r`	-
	`r|m`	`i'64`	-
`[LOCK] DEC` - Decrement [2, Page 3-326] Decrements the value in OP1 by 1	`r|m`	-	-
`MUL` - Unsigned Multiply [2, Page 4-148] Performs unsigned integer multiplication on `AL|AX|EAX|RAX` and OP1, and stores the result in `AX|DX:AX|EDX:EAX|RDX:RAX`	`r|m`	-	-
`IMUL` - Signed Multipy [2, Page 3-508] Multiplies signed integers. In the first encoding `AL|AX|EAX|RAX` is multiplied by OP1 and is stored in `AX|DX:AX|EDX:EAX|RDX:RAX`. In the second OP1 is multiplied by OP2, the result is truncated and stored in OP1. In the third OP1 is multiplied by the product of OP2 and OP3, the result is truncated and stored in OP1	`r|m`	-	-
	`r`	`r|m`	-
	`r`	`r|m`	`i'64`

Instruction	OP 1	OP 2	OP 3
`DIV` - Unsigned Divide [2, Page 3-328] Divides unsigned integers. It divides `AX|DX:AX|EDX:EAX|RDX:RAX` by OP1, stores the quotient in `AL|AX|EAX|RAX` and stores the remainder in `AH|DX|EDX|RDX`. Causes a Divide Error exception if OP1 is 0 or the quotient is larger than its destination register	r|m	-	-
`IDIV` - Signed Divide [2, Page 3-505] Divides signed integers. It divides `AX|DX:AX|EDX:EAX|RDX:RAX` by OP1, stores the quotient in `AL|AX|EAX|RAX` and stores the remainder in `AH|DX|EDX|RDX`. Causes a Divide Error exception if OP1 is 0 or the quotient is larger than its destination register	r|m	-	-
`[LOCK] NEG` - Two's Complement Negation [2, Page 4-165] Replaces OP1 with its two's complement (the same as subtracting it from 0)	r|m	-	-
Logical			
`BT` - Bit Test [2, Page 3-130] Stores the OP2 (mod OP1 size) bit in OP1 into CF	r|m	r|i8	-
`[LOCK] BTC` - Bit Test and Complement [2, Page 3-132] Stores the OP2 (mod OP1 size) bit in OP1 into CF and complements the bit in OP2	r|m	r|i8	-
`[LOCK] BTR` - Bit Test and Reset [2, Page 3-134] Stores the OP2 (mod OP1 size) bit in OP1 into CF and resets the bit (to 0) in OP2	r|m	r|i8	-
`[LOCK] BTS` - Bit Test and Set [2, Page 3-136] Stores the OP2 (mod OP1 size) bit in OP1 into CF and sets the bit (to 1) in OP2	r|m	r|i8	-
`[LOCK] XOR` - Logical Exclusive OR [2, Page 6-40] Performs a bitwise XOR on OP1 and OP2 and stores the result in OP1	r|m	i'64	-
	r|m	r	-
	r	r|m	-
`[LOCK] AND` - Logical AND [2, Page 3-75] Performs a bitwise AND on OP1 and OP2 and stores the result in OP1	r|m	i'64	-
	r|m	r	-
	r	r|m	-
`[LOCK] OR` - Logical Inclusive OR [2, Page 4-170] Performs a bitwise OR on OP1 and OP2 and stores the result in OP1	r|m	i'64	-
	r|m	r	-
	r	r|m	-
`[LOCK] NOT` - One's Complement Negation [2, Page 4-168] Performs a bitwise NOT on OP1 and stores the result in OP1	r|m	-	-

The condition codes (cc) are used in conditional instructions such as Jcc and replace the cc (not all codes are supported by all conditional instructions). Different codes check different flags or registers, the codes are described below along with their conditions:

Code	**Description**	**Flags**	**Signed-ness**
O	Overflow	OF = 1	
NO	Not Overflow	OF = 0	
S	Sign	SF = 1	
NS	Not Sign	SF = 0	
E	Equal	ZF = 1	
Z	Zero		
NE	Not Equal	ZF = 0	
NZ	Not Zero		
B	Below	CF = 1	Unsigned
NAE	Not Above or Equal		
C	Carry		
NB	Not Below	CF = 0	Unsigned
AE	Above or Equal		
NC	Not Carry		
BE	Below or Equal	CF = 1 or ZF = 1	Unsigned
NA	Not Above		
A	Above	CF = 0 and ZF = 0	Unsigned
NBE	Not Below or Equal		
L	Less	SF != OF	Signed
NGE	Not Greater or Equal		
GE	Greater or Equal	SF = OF	Signed
NL	Not Less		
LE	Less or Equal	ZF = 1 or SF != OF	Signed
NG	Not Greater		
G	Greater	ZF = 0 and SF = OF	Signed
NLE	Not Less or Equal		
P	Parity	PF = 1	
PE	Parity Even		
NP	Not Parity	PF = 0	
PO	Parity Odd		
CXZ	CX = 0		
ECXZ	ECX = 0		
RCXZ	RCX = 0		

[17]

6 Interrupts

Interrupts are signals from devices to the processor causing it to immediately pause execution and run code to handle the interrupt. When an interrupt occurs, the processor locates its entry in an OS provided lookup table called the Interrupt Descriptor Table (IDT) and can have up to 256 entries; once the processor locates the entry, it jumps to the code to handle the interrupt which is called the Interrupt Service Routine (ISR) or interrupt handler. The processor provides functionality to save processor state, if the ISR makes use of this then the interrupted process can be resumed without loss of continuity.

There are three types of interrupts:

1. Exceptions: Generated internally by the processor and are used to alert the kernel of an event which requires its attention such as a Double Fault, Page Fault, General Protection Fault, etc
2. Interrupt Request (IRQ) or Hardware Interrupt: Generated externally by devices on the chipset
3. Software Interrupt: Generated by software in order to get the kernels attention, often used for syscalls. The `INT` instruction is used to generate an interrupt

[18]

Each interrupt is generated as an interrupt vector which contains the interrupt vector number. Each interrupt condition which requires special handling by the processor is assigned a unique identification number (vector number). The first 32 vectors numbers are reserved by Intel for exceptions.

[3, Chapter 7.2]

There are three classifications of interrupt:

1. Faults - Generally correctable, allows the program to restart with no loss of continuity. Processor restores state prior to faulting instruction which is restarted when returned.
2. Traps - Reported immediately following the execution of a trapping instruction, allows the program to restart with no loss of continuity.
3. Aborts - Does not always report the precise location of the instruction, does not allow the restart of the program.

[3, Chapter 7.5]

Nonmaskable Interrupts (NMIs) are interrupts that cannot be masked (eg by EFLAGS.IF), furthermore subsequent NMIs are blocked until the current NMI handler returns.

[3, Chapter 7.7]

6.1 Sources of interrupts

Interrupts can be generated in two ways: hardware generated and software generated.

[3, Chapter 7.3]

The `INT` instruction is used to allow software to generate an interrupt by providing the interrupt vector number as an operand. The instruction is able to generate interrupts for the architecturally defined interrupts, however it does not push the error code onto the stack which would cause the ISR to return to an incorrect location. Furthermore calling `INT 2` will call the NMI handler without the processors NMI-handling hardware, furthermore software interrupts cannot be masked with EFLAGS.IF.

[3, Chapter 7.3.3] [3, Chapter 7.4.2]

Hardware interrupts generated by the processor:

Vector	Mnemonic	Type	Error Code	Description
0	#DE	Fault	No	Divide Error Exception
1	#DB	Fault/Trap	No	Debug Exception
2	-	Interrupt	No	NMI Interrupt
3	#BP	Trap	No	Breakpoint Exception
4	#OF	Trap	No	Overflow Exception
5	#BR	Fault	No	BOUND Range Exceeded Exception
6	#UD	Fault	No	Invalid Opcode Exception
7	#NM	Fault	No	Device Not Available Exception
8	#DF	Abort	Yes (zero)	Double Fault Exception
9	-	Fault	No	Coprocessor Segment Overrun
10	#TS	Fault	Yes	Invalid TSS Exception
11	#NP	Fault	Yes	Segment Not Present
12	#SS	Fault	Yes	Stack Fault Exception
13	#GP	Fault	Yes	General Protection Exception
14	#PF	Fault	Yes	Page-Fault Exception
16	#MF	Fault	No	x87 FPU Floating-Point Error
17	#AC	Fault	Yes	Alignment Check Exception
18	#MC	Abort	No	Machine-Check Exception
19	#XM	Fault	No	SIMD Floating-Point Exception
20	#VE	Fault	No	Virtualisation Exception
21	#CP	Fault	Yes	Control Protection Exception

Furthermore any vectors not listed in the range 0 to 31 are reserved and should not be used, all vectors in the range 32 to 255 are user defined interrupts.

[3, Chapter 7.3.1]

6.2 Interrupt Handling

Interrupt Service Routines (ISRs) are code that handle interrupts, they generally should preserve state of the processor by pushing any registers to the stack before using them and popping them off before returning to the previous process by using the `IRET` instruction.

The processor does not permit transfer of execution to a higher CPL, hence the handler should run at CPL 0.

[3, Chapter 7.12.1.2]

6.2.1 Protected Mode Handling

In protected mode, if there was no CPL change when calling the interrupt then the stack uses the interrupted program's stack, it pushes: EFLAGS, CS, EIP, then the Error Code (if there was an error code). If there was a CPL change then it switches the stack (SS and ESP for the handler are obtained from the TSS) and pushes to the new one: SS, ESP, EFLAGS, CS, EIP, then the Error Code (if there was an error code), furthermore in this case `IRET also restores the original stack`.

[3, Chapter 7.12.1]

Handlers can be either 16-bit or 32-bit and interrupt-stack-pushes are the same size.

Task gates can be used which cause a task switch to occur, this has the benefits of: automatically saving the entire context of the interrupted process, switching to a new CPL 0 stack, and handler can be isolated in separate address space (via a separate LDT). However it has worse performance and requires the handler to disable interrupts while the TSS is marked as busy (else another interrupt could attempt to use it and cause a #GP).

[3, Chapter 7.12.2]

6.2.2 IA-32e Mode Handling

In IA-32e mode, handlers are always 64-bit and interrupt-stack pushes are fixed at 64 bits (zero extended stores). SS:RSP always pushed, and SS is set to null on CPL change.

[3, Chapter 7.14]

Handlers stack is aligned to 16-bytes

[3, Chapter 7.14.2]

Old SS, RSP, RFLAGS, CS and RIP are pushed to the new stack.

The IST (interrupt stack table) provides up to seven IST pointers in the TSS, it is useful for executing specific interrupts (eg NMI, double-fault, machine-check) to execute on a known-good stack. Descriptors contain a 3-bit IST index that provides an offset into the IST section of the TSS. If the IST index is non-zero then the IST value at the index is loaded intp RSP. If the IST index is 0 then RSP is loaded with the RSP corresponding to the CPL in the TSS. SS is set to NULL and its RPL is set to new CPL.

[3, Chapter 7.14.5]

6.2.3 Error Codes

Some exceptions push an error code to the stack after their return address.

Format of the error code:

Bits	Label	Description
31:16	Rsvd.	Must be 0
15:3	Selector	Segment Selector Index
2	TI	When set it indicated the selector references the LDT, otherwise it references the LDT (Only used when IDT is clear)
1	IDT	When set it indicates the selector references the IDT, otherwise it references the GDT/LDT
0	EXT	When set it occured due to an external event, if cleared it was caused by an `INT` instruction

Format is different for #PF and #CF, and reserved portion may not be pushed in order to align the stack. IRET does not pop the error code off the stack so ISR must do it.

[3, Chapter 7.13]

6.3 PIC

IBM-PC compatible environments contain a PIC (Programmable Interrupt Controller). The PIC has been replaced by the APIC which should be used instead.

Most systems contain two PICs, each with 8 inputs and a single output; the slave PIC's output is connected to the master PIC's third input (input #2). When the slave PIC attempts to interrupt the processor, it sends it to the master PIC which then interrupts the processor, this means that IRQ #2 cannot occur. Each PIC has an 8-bit Interrupt Mask Register (IMR), when a bit is set the PIC ignores the request on that line.

External devices send a signal to the PIC when they need to interrupt the processor (without the PIC the processor would have to poll these devices). The PIC translates the IRQ number (from the interrupt line the signal was on) into an interrupt vector, where it then interrupts the processor. On system start up, IRQs 0:7 are set to interrupts 0x08:0x0F and IRQs 8:15 are set to interrupts 0x70:0x77 by the BIOS. IQRs are handled by priority level: 0, 1, 2, 8, 9, 10, 11, 12, 13, 14, 15, 3, 4, 5, 6, 7. The ISR handles the interrupt by communicating with the device via the `in` and `out` instructions, once the ISR has finished it should inform the PIC that it can send new interrupts. In order to inform the master PIC, 20h should be sent to port 20h; to inform the slave PIC, 20h should be send to port A0h and port 20h.

The PICs can be remapped so IRQs are mapped to different interrupt vectors (useful since the default interrupts for several IRQs are reserved), however they can only be mapped to multiples of 8. To remap/initialise the PIC, send it 0x11 to begin initialisation on its command port, then on the data port send it: the vector offset, then 0x04 if its the master and 0x02 if its the slave, finally send it 0x01 (operate in 8086 mode).

Standard ISA IRQs:

IRQ	Description
0	Programmable Interrupt Timer Interrupt
1	Keyboard Interrupt
2	Cascade (used internally by the PICs, never raised)
3	COM2 (if enabled)
4	COM1 (if enabled)
5	LPT2 (if enabled)
6	Floppy disk
7	LPT1 / Unreliable spurious interrupt
8	CMOS real-time clock (if enabled)
9	Free for peripherals / legacy SCSI / NIC
10	Free for peripherals / SCSI / NIC
11	Free for peripherals / SCSI / NIC
12	PS2 Mouse
13	FPU / Coprocessor / Inter-processor
14	Primary ATA Hard Disk
15	Secondary ATA Hard Disk

Ports used by PIC:

Port	Description
20h	Master PIC control
21h	Master PIC mask/data
A0h	Slave PIC control
A1h	Slave PIC mask/data
60h	Keyboard controller data
64h	Keyboard controller command

Example keyboard ISR:

```
push eax ; Save state
in al, 60h ; Read from keyboard (al contains scancode)
mov al, 20h
out 20h, al ; Acknowledge interrupt
pop eax ; Restore state
iret ; Return from ISR
```

[18] [19]

6.4 APIC

The APIC (Advanced Programmable Interrupt Controller) is the updated standard for the older PIC. The APIC is used for sophisticated interrupt redirection and for sending interrupts between processors in multiprocessor systems. When using the APIC the PICs should be disabled, this can be done by sending 0xFF to both PIC mask ports. The PIC should still be remapped as it could still give spurious interrupts which would then be misinterpreted by the OS. There is both a local APIC and I/O APIC, the I/O APIC is an external chip whilst there is a local APIC in each processor (this document focuses on the local APIC).

APIC ID is returned in EBX[31:24] by CPUID 1

[3, Chapter 12.4.6]

The Local APIC contains registers, which are memory mapped in a single 4-KiB space. They are 32-bits registers and are 128-bit aligned and should be accessed using 32-bit load/stores. The memory between registers should not be accessed, and wider registers are accessed via multiple 32-bit accesses. By default it is a memory-mapped to a 4-KiB region starting at 0xFEE00000 and should be designated as strong uncacheable (UC). It can be relocated using the IA32_APIC_BASE MSR

The following is a list of the APIC registers along with their offsets to the APIC base address:

Offset	**Register Name**	**R/W**
000H	Rsvd.	
010H	Rsvd.	
020H	Local APIC ID Register	RW
030H	Local APIC Version Register	R
040H	Rsvd.	
050H	Rsvd.	
060H	Rsvd.	
070H	Rsvd.	
080H	Task Priority Register (TPR)	RW
090H	Arbitration Priority Register (APR)	R
0A0H	Processor Priority Register (PPR)	R
0B0H	EOI Register	W
0C0H	Remote Read Register (RRD)	R
0D0H	Logical Destination Register	RW
0E0H	Destination Format Register	RW
0F0H	Spurious Interrupt Vector Register	RW
100H	In-Service Register (ISR); bits 31:0	R
110H	In-Service Register (ISR); bits 63:32	R
120H	In-Service Register (ISR); bits 95:64	R
130H	In-Service Register (ISR); bits 127:96	R
140H	In-Service Register (ISR); bits 159:128	R
150H	In-Service Register (ISR); bits 191:160	R
160H	In-Service Register (ISR); bits 223:192	R
170H	In-Service Register (ISR); bits 255:224	R
180H	Trigger Mode Register (TMR); bits 31:0	R
190H	Trigger Mode Register (TMR); bits 63:32	R
1A0H	Trigger Mode Register (TMR); bits 95:64	R
1B0H	Trigger Mode Register (TMR); bits 127:96	R
1C0H	Trigger Mode Register (TMR); bits 159:128	R
1D0H	Trigger Mode Register (TMR); bits 191:160	R
1E0H	Trigger Mode Register (TMR); bits 223:192	R
1F0H	Trigger Mode Register (TMR); bits 255:224	R

Offset	Register Name	R/W
200H	Interrupt Request Register (IRR); bits 31:0	R
210H	Interrupt Request Register (IRR); bits 63:32	R
220H	Interrupt Request Register (IRR); bits 95:64	R
230H	Interrupt Request Register (IRR); bits 127:96	R
240H	Interrupt Request Register (IRR); bits 159:128	R
250H	Interrupt Request Register (IRR); bits 191:160	R
260H	Interrupt Request Register (IRR); bits 223:192	R
270H	Interrupt Request Register (IRR); bits 255:224	R
280H	Error Status Register	RW
290H	Rsvd.	
2A0H	Rsvd.	
2B0H	Rsvd.	
2C0H	Rsvd.	
2D0H	Rsvd.	
2E0H	Rsvd.	
2F0H	LVT Corrected Machine Check Interrupt (CMCI) Register	RW
300H	Interrupt Command Register (ICR); bits 31:0	RW
310H	Interrupt Command Register (ICR); bits 63:31	RW
320H	LVT Timer Register	RW
330H	LVT Thermal Sensor Register	RW
340H	LVT Performance Monitoring Counters Register	RW
350H	LVT LINT0 Register	RW
360H	LVT LINT1 Register	RW
370H	LVT Error Register	RW
380H	Initial Count Register (for Timer)	RW
390H	Current Count Register (for TImer)	R
3A0H	Rsvd.	
3B0H	Rsvd.	
3C0H	Rsvd.	
3D0H	Rsvd.	
3E0H	Divide Configuration Register (for Timer)	RW
0F0H	Rsvd.	

[3, Chapter 12.4.1]

The following table describes the layout of the Local Vector Tables (LVTs) Registers:

LVT	31:19	18:17	16	15	14	13	12	11	10:8	7:0
Timer	Rsvd.	TM1	Mask	Rsvd.	Rsvd.	Rsvd.	DS	Rsvd.	Rsvd.	Vector
CMCI	Rsvd.	Rsvd.	Mask	Rsvd.	Rsvd.	Rsvd.	DS	Rsvd.	DM	Vector
LINT0	Rsvd.	Rsvd.	Mask	TM2	RIRR	IIPP	DS	Rsvd.	DM	Vector
LINT1	Rsvd.	Rsvd.	Mask	TM2	RIRR	IIPP	DS	Rsvd.	DM	Vector
Error	Rsvd.	Rsvd.	Mask	Rsvd.	Rsvd.	Rsvd.	DS	Rsvd.	Rsvd.	Vector
Perf Counters	Rsvd.	Rsvd.	Mask	Rsvd.	Rsvd.	Rsvd.	DS	Rsvd.	DM	Vector
Thermal Sensor	Rsvd.	Rsvd.	Mask	Rsvd.	Rsvd.	Rsvd.	DS	Rsvd.	DM	Vector

The following list describes the use of several of the fields in the previous table:

- Timer Mode (TM1):
  00 = One-shot, 01 = Periodic, 10 = TSC-Deadline (uses the absolute target value in the IA32_TSC_DEADLINE MSR)
- Mask:
  0 = Not Masked, 1 = Masked. When the performance counter interrupt is generated, its mask bit is set.
- Vector:
  Interrupt Vector Number
- Delivery Status (DS) - Read only:
  0 = Idle, 1 = Send Pending
- Delivery Mode (DM):
  - 000 = Fixed - delivers the interrupt in the vector field
  - 010 = SMI - delivers a SMI interrupt, vector should be set to 0
  - 100 = NMI - delivers a NMI interrupt, vector is ignored
  - 101 = INIT - delivers an INIT request, vector should be set to 0, not supported by CMCI, thermal monitor or performance counter
  - 111 = ExtINT - causes the processor to respond to the interrupt as if it originated from an externally connected (8259A-compatible) interrupt controller, a INTA signal is sent to the external controller which is expected to provide the vector information, only one processor should contain an ExtINT and there should only be one ExtINT source. Not supported by CMCI, thermal monitor or performance counter
- Trigger Mode (TM2):
  0 = Edge sensitive, 1 = Level sensitive. Only used when delivery mode is fixed otherwise it is always edge sensitive, unless the delivery mode is ExtINT then it is level sensitive. Timer and error interrupts are treated as edge sensitive. If the local APIC is not used with an I/O APIC and fixed delivery mode is used then it will always use level sensitive mode. Level-sensitive interrupts are not supported for LINT1
- Remote IRR (RIRR) - Read only:
  For fixed-mode level-triggered interrupts, this flag is set then the local APIC accepts the interrupt for servicing and is reset when an EOI command is received from the processor
- Interrupt Input Pin Polarity (IIPP):
  0 = Active high, 1 = Active low

[3, Chapter 12.5.1]

For all interrupts except those delivered with the NMI, SMI, INIT, ExtINT, start-up or INIT-deassert, the interrupt handler must write 0s to the end-of-interrupt (EOI) register, this must occur at the end of the interrupt handler before the IRET. This indicates the interrupt is complete and the local APIC can issue another interrupt.

[3, Chapter 12.8.5]

6.4.1 Priority

Each interrupt delivered to the processor through the local APIC has a priority based on its vector number. The interrupt-priority class is the value of bits 7:4 of the vector number. The lowest priority class is 1 and the highest is 15, vector numbers 0-15 are illegal so are never delivered, since vectors 0-31 are reserved, software should configure interrupt vectors to use interrupt-priority classes 2-15. Each interrupt class encompasses 16 vectors, the relative priority of these vectors is determined by bits 3:0 of the interrupt vector.

[3, Chapter 12.8.3]

6.4.2 Errors

The local APIC records errors detected during interrupt handling in the Error Status Register (ESR).

Error Status Register Format:

Bits	Description
31:8	Rsvd.
7	Illegal Register Address
6	Received Illegal Vector
5	Send Illegal Vector
4	Redirectable IPI
3	Receive Accept Error
2	Send Accept Error
1	Receive Checksum Error
0	Send Checksum Error

Before reading from the ESR, software should write 0s to it, this updates it with any errors detected since the last write to it. The LVT Error Register specifies the vector of the interrupt to be delivered when the error was detected.

[3, Chapter 12.5.3]

6.4.3 Interprocessor Interrupts

Interprocessor Interrupts (IPIs) are used to send interrupts to other processors, usually for the following function:

- Send an interrupt to another processor
- Allow the processor to forward an interrupt to another processor to be serviced
- To direct the processor to interrupt itself
- To deliver special IPIs such as the start-up IPI (SIPI) message to other processors

The Interrupt Command Register (ICR) is used to program the IPI.

[3, Chapter 12.6]

Interrupt Command Register:

Bits	Name
63:56	Destination Field
55:20	Rsvd.
19:18	Destination Shorthand
17:16	Rsvd.
15	Trigger Mode
14	Level
13	Rsvd.
12	Delivery Status
11	Destination Mode
10:8	Delivery Mode
7:0	Vector

Description of fields used in the ICR:

- Destination Field:
  Only used when shorthand is 00, if the destination mode is physical then it is the APIC ID of the target processor otherwise if it is logical its interpretation depends on the settings of the DFR and LDR registers of the local APICS in all the processors
- Destination Shorthand:
  00 = No shorthand, 01 = Self, 10 = All including self, 11 = All excluding self
- Trigger Mode:
  0 = Edge, 1 = Level. Only used when using the INIT level de-assert deliver mode
- Level:
  0 = De-assert, 1 = Assert. For INIT level de-assert delivery mode this flag must be set to 0, otherwise it must be set to 1
- Delivery Status (Read-Only):
  0 = Idle, 1 = Send Pending
- Destination Mode:
  0 = Physical, 1 = Logical
- Delivery Mode:
  - 000 = Fixed - Delivers the interrupt in vector to the processor
  - 001 = Lowest Priority - Same as fixed but uses the lowest priority. Model specific, do not use
  - 010 = SMI - Delivers an SMI interrupt, vector should be 0
  - 011 = Rsvd.
  - 100 = NMI - Delivers an NMI interrupt, vector is ignored
  - 101 = INIT - Sends INIT request to the processor which causes it to perform an INIT, vector should be 0. If level = 0 and trigger = 1, sends a sync message to all local APICs to set their arbitration IDs to their APIC IDs, software should still specify shorthand = 10
  - 110 = Start Up - Sends Start-up IPI (SIPI), vector points to start-up routine. It is software's responsitiblity to detect errors in sending
  - 111 = Rsvd.
- Vector:
  vector number of the interrupt being sent

Writing to the lower ICR register causes the IPI to be sent.

[3, Chapter 12.6.1]

In logical destination mode, the Logical Destination Register (LDR) contains the Logical APIC ID in its highest byte (bits 31:24). The Destination Format Register (DFR) selects between two different logical models, 0xFFFFFFFF selects the flat model (default) while 0x0FFFFFF selects the cluster model.
In the flat model, each local APIC performs a bitwise AND between its logical APIC ID and the IPI's destination field, it it is non-zero then the local APIC accepts the IPI.

[3, 12.6.2.2]

6.4.4 Timer

The local APIC contains a programmable 32-bit timer. Its frequency is either bus clock or crystal clock frequency (when it is enumerated in CPUID 15H) divided by the value in the Divide Configuration Register. The timer can be programmed through the LVT Timer register for one-shot or periodic operation. The timer is started by programming the Initial Count Register, which is copied to the Current Count Register and count down begins, when it reaches zero the timer interrupt is generated. In one-shot mode when the timer reaches zero it is not restarted, in periodic mode it is restarted with the value in the Initial Count Register. Writing to the Initial Count Register restarts the timer with the new value (even if it is currently counting down), writing zero to it stops the timer.

Divide configuration Register (bits 3, 1, 0):

Bits	000	001	010	011	100	101	110	111
Divide	2	4	8	16	32	64	128	1

Default divide is 2

[3, Chapter 13.5.4]

6.4.5 Spurious Interrupt

When a processor raises its task priority to be greater than or equal to the level of the interrupt for which the processor INTR signal is currently being asserted. If at the time the INTA cycle is issued the interrupt that was dispensed has become masked, the APIC will deliver a spurious-interrupt vector, it does not affect the ISR so the handler should not return an EOI.

Spurious-Interrupt Vector Register (SVR):

Bits	**Description**
31:13	Rsvd.
12	EOI-Broadcast Supression, 0 = Broadcast EOI, 1 = Do not broadcast EOI
11:10	Rsvd.
9	Focus Processor Checking, 0 = Enabled, 1 = Disabled
8	0 = APIC Disabled, 1 = APIC Enabled
7:0	Spurious Vector

The Spurious vector is the interrupt vector number to be delivered when the APIC generates a spurious interrupt. On some processors bits 3:0 are 1.

[3, Chapter 12.9]

6.5 Interrupt Descriptor Table

The Interrupt Descriptor Table (IDT) associates each interrupt vector with an interrupt descriptor for the ISR to handle the interrupt. The linear base address of the IDT and limit (size of IDT - 1) is contained in the IDT register (IDTR), which can be written to using the `SIDT` instruction. Since there are only 256 interrupt vectors, the IDT should not contain more than 256 descriptors. In protected mode each descriptor is 8 bytes and in IA-32e mode each descriptor is 16 bytes. When an interrupt occurs the interrupt vector provides a vector number which is used to index into the IDT, and uses the descriptor to call the corresponding ISR handler.

[3, Chapter 7.10]

In protected mode there are three types if interrupt descriptor, whilst in IA-32e mode there are only two.

1. Task-gate descriptor (protected mode only)
2. Interrupt-gate descriptor (both)
3. Trap-gate descriptor (both)

The only difference between interrupt gates and trap gates is that interrupt gates clears EFLAGS.IF which prevents other interrupts from occurring while the ISR is running, whilst trap gates do not.

[3, Chapter 7.12.1.3]

Protected mode Task-gate descriptor:

Bits	Label	Description
63:48	Rsvd.	Must be 0
47	P	Segment Present Flag
46:45	DPL	Descriptor Privilege Level
44:40	00101	Must be 00101
39:32	Rsvd.	Must be 0
31:16	CS	TSS Segment Selector
15:0	Rsvd.	Must be 0

Protected mode Interrupt/Trap-gate descriptor:

Bits	Label	Description
63:48	Offset 31:16	Bits 31:16 of the address of the ISR
47	P	Segment Present Flag
46:45	DPL	Descriptor Privilege Level
44:40	Type	Segment Type
39:32	Rsvd.	Must be 0
31:16	CS	Code Segment Selector
15:0	Offset 15:0	Bits 15:0 of the address of the ISR

Where type is 00110 for a 16-bit interrupt gate, 01110 for a 32-bit interrupt gate, 00111 for a 16-bit trap gate, or 32-bit trap gate.

[3, Chapter 7.11]

IA-32e Interrupt/Trap gate descriptor:

Bits	Label	Description
127:96	Rsvd.	Reserved (must be 0)
95:64	Offset 63:32	Bits 63:32 of the address of the ISR
63:48	Offset 31:16	Bits 31:16 of the address of the ISR
47	P	Segment Present Flag
46:45	DPL	Descriptor Privilege Level
44:35	Type	Descriptor type
34:32	IST	Interrupt Stack Table
31:16	CS	Code Segment Selector
15:0	Offset 15:0	Bits 15:0 of the address of the ISR

Where type is 0111100000 for an Interrupt Gate or 0111000000 for a Trap Gate

[3, Chapter 7.14] [3, Chapter 3.5]

6.6 Real Mode Interrupts

In real mode, interrupts are handled similarly to how they are handled in protected mode. It utilises the interrupt vector table, each entry in the table is 32-bits; the first two bytes stores the offset and the second two stores the CS of the interrupt handler. The base address of the table is at physical address 0 and its limit is 3FFH. When and interrupt is called it pushes the CS, IP and the low 16 bits of EFLAGS onto the stack, then indexes into the interrupt table using the interrupt vector and calls the handler, the handler returns by using the `IRET` instruction which restores program state.

[3, 22.1.4]

6.7 BIOS Interrupts

6.8 Syscalls

In order for a program to request resources from the OS it has to hand over control of the processor to the OS via a syscall/interrupt. 32-bit programs use the interrupt `int 80h`, while 64-bit programs use the instruction `syscall`. The parameters for the syscall are placed in specific registers, the following is a list of registers used and their use in syscalls:

i386 Registers	x86-64 Registers	Usage
EBX	RDI	Arg1
ECX	RSI	Arg2
EDX	RDX	Arg3
ESI	R10	Arg4
EDI	R8	Arg5
EBP	R9	Arg6
EAX	EAX	Syscall
EAX	RAX	Return

Furthermore, the values in these registers (and others such as R11 and RCX) may be clobbered (their value changing after the syscall) so it is best to follow the System V ABI when performing syscalls. When using the syscall function in C if the syscall had a negative return (error), it is negated and copied to errno, and the function returns -1.

[20]

The following table contains the arguments, descriptions, return information, and syscall number (for both i386[21] and x86-64[22]) for various common Linux syscalls:

Name	Return	i386	x86-64	Arguments
`read[23]`	`length|error`	`3`	`0`	1: fd 2: buf* 3: count
Attempts to read up to `count` bytes from the current position of the file referred to by `fd` into the buffer pointed to by `buf`. The true number of bytes read is returned (if EOF less bytes would be read) and the position in the file is advanced by this many bytes, otherwise returns an error				
`write[24]`	`length|error`	`4`	`1`	1: fd 2: buf* 3: count
Writes up to `count` bytes from `buf` into the file referred to by `fd` at the current position of the file, and its position is advanced by the number of bytes written. On success the number of bytes written is returned, otherwise returns an error				
`open[25]`	`fd|error`	`5`	`2`	1: path* 2: flags 3: mode
Opens the file located at `path` using the given `flags`. On success returns the file descriptor (`fd`), otherwise returns an error. The mode argument is for the permissions of the created file when O_CREAT or O_TMPFILE is given as a flag and is otherwise ignored				
`close[26]`	`0|error`	`6`	`3`	1: fd
Closes the file descriptor (`fd`). On success returns 0, otherwise returns an error				
`fstat[27]`	`0|error`	`108`	`5`	1: fd 2: statbuf*
Writes information about the file referred to by `fd` into the `struct stat[30]` pointed to by `statbuf`. On success returns 0, otherwise returns an error				
`exit[28]`	`noreturn`	`1`	`60`	1: status
Terminates the calling thread, `status & 0xFF` is returned to the parent process (if the thread was the last in the process)				
`munmap[29]`	`0|error`	`91`	`11`	1: addr* 2: length
All pages containing part of the specified address range are unmapped (the region is also unmapped when the process is terminated). `addr` must be a multiple of page size but `length` does not need to be. On success 0 is returned, otherwise an error is returned (it is not an error if there were no mapped pages within the range)				

Name	Return	i386	x86-64	Arguments
mmap[29]	addr*|error	90	9	1: addr* 2: length 3: prot 4: flags 5: fd 6: offset
Creates a new mapping in the virtual address space of the process. The kernel attempts to create a mapping aligned to a page boundary close to the `addr`, or chooses one itself if `addr` is NULL. The contents of a file mapping are initialised using `length` bytes starting at `offset` (multiple of page size) in the file referred to by `fd`, the file descriptor can be closed after mmap has returned without invalidating the mapping. `prot` describes the memory protection of the mapping (must not conflict with the open mode of the file); it is a bitwise OR of READ (1), WRITE(2) and EXEC (4). `flags` determines if updates to the mapping are visible to other processes with mappings to the same region, and if they are carried through to the underlying file; one important flag is MAP_ANONYMOUS (0x20) for mappings not backed by files, `fd` should be -1 and `offset` should be 0. On success the address of the mapping is returned, otherwise an error is returned				

A fd (file descriptor) in Linux is an integer used to refer to a file, when opening a file the kernel returns an integer to the program to refer to the file with (lowest unused integer) and the kernel keeps track of the open file. Integers 0, 1 and 2 refer to stdin, stdout and stderr respectively; any negative fd is invalid -1 is usually used to refer to no file[31]. Furthermore any arguments in the table which contain a * such as buf* are pointers.

7 Appendix

All of the code used/written has been appended below, it includes programs that can run on Linux or baremetal

7.1 Bash scripts

These bash scripts were used to run the code.

64-bit Linux code (Links together files if given more than one argument)

```bash
#!/bin/bash

# Assembles the assembly files it is given into object files in the same dir
# and name as the file it was given. Links together all object files given (and
# the ones it assembled) into a binary file in the directory and name of the
# first argument. If given '--' as an argument, all arguments after it will be
# passed to the executeable.

if [ $# -eq 0 ]; then
    echo "Must have at least one argument"
    exit 1
fi

objects=()
args=()
is_arg=false

for arg in "${@}"; do
    if [ $is_arg = true ]; then
        args+=($arg)
    elif [[ $arg = "--" ]]; then
        if [[ ${#objects[@]} = 0 ]]; then
            echo "Must give at least one file before the program arguments"
            exit 1
        else
            is_arg=true
        fi
    else
        file="$(pwd)/$arg"

        case $file in
            *.asm | *.s | *.S)
                object="${file%.*}.o"
                objects+=($object)
                nasm -f elf64 $file -o $object || exit 1 ;;
            *.o | *.a | *.so)
                objects+=($file) ;;
            *)
                echo "'$file' does not end in a correct file extention"
                exit 1 ;;
```

```
        esac
    fi
done

output="$(pwd)/${1%.*}.bin"
ld "${objects[@]}" -o $output || exit 1
$output "${args[@]}"
```

32-bit Linux code

```
#!/bin/bash

if [ $# -ne 1 ]; then
  echo "Incorrect Args"
  exit 1
fi

ASM="$(pwd)/$1"
if [[ $ASM == *.asm ]]; then
  BASE="${ASM%.asm}"
else
  echo "File does not end in .asm"
  exit 1
fi

OBJECT="${BASE}.o"

nasm -f elf32 $ASM -o $OBJECT
ld -m elf_i386 $OBJECT -o $BASE
$BASE
```

7.2 Hello world (Linux)

This program writes "Hello World" to stdout then exits without an error.

64-bit code:

```
section .data
    hello db 'Hello world', 0x0A ; Hello world followed by a newline
    ; $ is current mem location, then subtracts start of hello to get length
    length equ $ - hello ; Length of hello in bytes

section .text
    global _start ; Make _start accessible to the linker

_start:
    mov rax, 1 ; set syscall to 1 (sys_write)
    mov rdi, rax ; set arg1 to 1 (stdout)
    mov rsi, hello ; set arg2 to point to hello
    mov rdx, length ; set arg3 to length of hello
    syscall ; perform the syscall

    mov rax, 60 ; set syscall to 60 (exit)
    xor rdi, rdi ; set arg1 to 0 (success)
    syscall ; perform the syscall
```

32-bit code:

```
section .data
    hello db 'Hello world', 0x0A
    length equ $ - hello

section .text
    global _start

_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, hello
    mov edx, length
    int 80h

    mov eax, ebx
    xor ebx, ebx
    int 80h
```

7.3 Difference (Linux)

This program reads two integers from stdin and writes their difference to stdout

```
section .bss
    print_buffer_len equ 128
    print_buffer resb print_buffer_len ; Reserve bytes of uninitilised memory

section .data
    prompt_text db 'Enter two values to find their difference', 0x0A
    prompt_len equ $ - prompt_text

section .text
    global _start

_start:
    ; print prompt text
    mov rax, 1
    mov rdi, rax
    mov rsi, prompt_text
    mov rdx, prompt_len
    syscall

    call read_rax
    mov rbx, rax
    call read_rax
    cmp rax, rbx
    jae skip_exchange
    xchg rax, rbx
skip_exchange:
    sub rax, rbx

    call print_rax
exit:
    xor rdi, rdi
    mov rax, 60
    syscall

; returns rax (value read)
read_rax:
    push rdi
    push rsi
    push rbx
    push rcx
    push rdx

    xor rax, rax
    xor rdi, rdi
    mov rsi, print_buffer
    mov rdx, print_buffer_len
    syscall

    ; Exit if returned value is greater than the buffer length
    cmp rax, print_buffer_len
    ja exit

    mov rcx, rax
    xor rax, rax
    mov rdi, 10
    xor rbx, rbx
read_loop:
    ; Exit if reached end of buffer and no newline
    dec rcx
    jc exit

    mov byte bl, [rsi]
    cmp rbx, 0x0A
    je read_return ; return if newline

    sub rbx, 48
    cmp rbx, 9
    ja exit ; exit if above 9 (so rbx is between 0 and 9 inclusive)

    mul rdi
    add rax, rbx
    inc rsi
    jmp read_loop
```

```

read_return:
    pop rdx
    pop rcx
    pop rbx
    pop rsi
    pop rdi
    ret

print_rax:
    push rdi
    push rsi
    push rdx
    push rbx
    push rax

    mov rsi, print_buffer
    xor rdi, rdi
    mov rbx, 10
print_loop_right:
    inc rdi
    xor rdx, rdx
    div rbx
    cmp rax, 0
    jne print_loop_right
    ; rsi now points to the location where the newline character should be
    add rsi, rdi
    mov byte [rsi], 0x0A

    ; read rax again
    mov rax, [rsp]
    inc rdi ; rdi now stores len of string
print_loop_left:
    dec rsi
    xor rdx, rdx
    div rbx
    add rdx, 48
    mov byte [rsi], dl
    cmp rax, 0
    jne print_loop_left

    mov rdx, rdi
    mov rax, 1
    mov rdi, rax
    syscall

    pop rax
    pop rbx
    pop rdx
    pop rsi
    pop rdi

    ret
```

7.4 Cat (Linux)

This program performs the cat command (it takes in one argument which is a filename and prints its contents). It uses the second arg (the first being the path) as the filename for the open syscall, it then calls the fstat syscall on the returned file descriptor. It gets the size of the file from fstat and allocates the file to memory (the size allocated is the next multiple of page size of the file size). It then writes the memory to stdout then deallocates the memory, closes the file and exits. It also writes to stderr and exits on critical errors (ones that would cause undefined behaviour) but ignores non-critical errors

```
; Cli args (stack)
; Syscalls: open, fstat, mmap, write, munmap, close, exit

section .bss
    stat_buffer resb stat_buffer_len ; size of the struct stat

section .data
    arg_error_text db 'Must be given a single argument', 0x0A
    arg_error_len equ $ - arg_error_text
    open_error_text db 'Failed to open file', 0x0A
    open_error_len equ $ - open_error_text
```

```
    fstat_error_text db 'Failed to call fstat', 0x0A
    fstat_error_len equ $ - fstat_error_text
    mmap_error_text db 'Failed to call mmap', 0x0A
    mmap_error_len equ $ - mmap_error_text
    stat_buffer_len equ 144

section .text
    global _start

_start:
    cmp qword [rsp], 2 ; [rsp] contains no. of args
    jne arg_exit
    mov rdi, [rsp+16] ; [rsp+8] contains ptr to path arg, [rsp+16] contains first arg

    ; open file, filename already in rdi, rdx is garbage as O_CREAT is not set
    xor rsi, rsi ; 0 for O_RDONLY (read only)
    mov rax, 2
    syscall
    cmp rax, 0
    jl open_exit ; exit if fd is negative (error)

    ; call fstat
    mov rdi, rax ; fd
    mov rsi, stat_buffer
    mov rax, 5
    syscall
    cmp rax, 0
    jne fstat_exit ; exit if an error is returned

    mov r8, rdi ; fd
    ; move size of file into rsi and round it up to a multiple of page size (4KiB)
    mov qword rsi, [stat_buffer+48]
    add rsi, 4096
    mov rax, 0xFFF ; Load mask
    not rax ; Not mask (all 1s but the 1s less than 4096)
    and rsi, rax
    mov r12, rsi ; store in r12 so its not overwritten

    ; Allocate memory contained by fd
    xor rdi, rdi ; NULL for any starting address
    mov rdx, 1 ; PROT_READ (prot)
    mov r10, 2 ; MAP_PRIVATE (flags)
    xor r9, r9 ; 0 offset
    mov rax, 9
    syscall
    cmp rax, 0
    jl mmap_exit ; exit if an error is returned

    mov rsi, rax ; ptr to address
    mov rdx, [stat_buffer+48]
    mov rdi, 1 ; stdout
    mov rax, rdi
    syscall ; print buffer

    ; munmap buffer (not required as we exit anyway)
    mov rdi, rsi
    mov rsi, r12
    mov rax, 11
    syscall ; Ignore potential error since we exit anyway

    ; close file (not required as we exit anyway)
    mov rdi, r8
    mov rax, 3 ; line could be removed, the fd is likely 3 as its the lowest unused fd
    syscall ; Ignore potential error since we exit anyway

    ; exit
    xor rdi, rdi
    mov rax, 60
    syscall

mmap_exit:
    mov rsi, mmap_error_text
    mov rdx, mmap_error_len
    jmp exit
fstat_exit:
    mov rsi, fstat_error_text
```

```
    mov rdx, fstat_error_len
    jmp exit
open_exit:
    mov rsi, open_error_text
    mov rdx, open_error_len
    jmp exit
arg_exit:
    mov rsi, arg_error_text
    mov rdx, arg_error_len
exit: ; rsi - ptr, rdx - len
    ; print to stderr
    mov rdi, 2
    mov rax, 1
    syscall
    ; exit
    mov rdi, 1
    mov rax, 60
    syscall
```

7.5 Hello world (Baremetal)

This program prints "Hello world" to the vga-mode buffer.

```
BITS 16                     ; Instruct NASM to produce 16 bit (real mode) code
org 0x7c00                  ; Set the origin to 0x7c00 which is where BIOS loads the bootloader

start:
    mov ax, 0xb800          ; Video memory segment (0xb800)
    mov es, ax              ; Load ES with the video memory segment

    xor di, di              ; Start of memory segment (using di as an offset)
    mov cx, 2000            ; 80 by 25 chars

clearscreen:
    mov byte [es:di], 0x20  ; Write space to video memory at ES:DI
    inc di

    mov byte [es:di], 0x07  ; Write attribute (light gray on black) to video memory at ES:DI
    inc di

    dec cx
    jnz clearscreen         ; Repeat if cx is non-zero

    lea si, [message]       ; Load the address of 'message' into SI
    xor di, di              ; Start at the beginning of the video memory (offset 0)
    mov cx, message_len

print_line:
    mov byte al, [si]       ; Move byte from message into the video memory
    mov byte [es:di], al

    dec cx
    jz halt

    inc si
    add di, 2

    jmp print_line

halt:
    cli                     ; Disable interrupts
    hlt                     ; Halt (Only resumes on an interrupt however they were disabled)

message db 'Hello world'
message_len equ $ - message

; Boot sector padding and signature
times 510-($-$$) db 0       ; Pad the boot sector to 510 bytes (ensuring the total size is 512 bytes
    )
dw 0xAA55                   ; Boot sector signature (0xAA55), required for a valid bootable sector
```

7.6 Kernel (Baremetal)

This program loads a (very) basic 64-bit kernel, it is capable of booting into a 64-bit environment using identity-mapped memory and can handle PS/2 keyboard interrupts. It starts execution in the 16-bit assembly, which sets up a 32-bit environment and calls __start__ in bootloader.c which sets up the 64-bit environment, returns to the assembly code to enable 64-bit mode then jumps to __start__ in kernel.c.
This bash script is used to compile and run the kernel (using qemu)

```
#!/bin/bash

location=$(dirname $(realpath $0))
src="${location}/src"
link="${location}/link"
build="${location}/build"
boot="${build}/bootloader.bin"
image="${build}/image.bin"
nasm -f bin "${location}/bootloader.asm" -o "$boot" || exit 1
dd if=/dev/zero "of=$image" bs=512 count=2880 || exit 1
dd "if=$boot" "of=$image" bs=512 count=1 conv=notrunc || exit 1

o32=()

while read -r line; do
    cfile="${src}/$line"
    ofile="${build}/$line"
    ofile="${ofile%.*}.o"
    o32+=("$ofile")
    gcc -fno-stack-protector -m32 -ffreestanding -nostdlib -nostartfiles -fno-ident -fno-asynchronous-unwind-tables -c "$cfile" -o "$ofile" || exit 1
  # ^ Disable stack protector so we can return from _start
done < "${link}/link32" || exit 1

output="${build}/kernel32.bin"
elf="${build}/kernel32.elf"
ld -m elf_i386 -T "${link}/ld32" --entry=_start "${o32[@]}" -o "$elf" || exit 1
objcopy -O binary -R .note.gnu.property -R .note.gnu.build-id "$elf" "$output"

if [ $(stat -c%s "$output") -gt 4096 ]; then # Change to whatever the number of sectors * 512 the bootloader is reading
    echo "${output} file is too large"
    exit 1
fi

dd "if=$output" "of=$image" bs=512 seek=1 conv=notrunc || exit 1

o64=()

while read -r line; do
    cfile="${src}/$line"
    ofile="${build}/$line"
    ofile="${ofile%.*}.o"
    o64+=("$ofile")
    gcc -m64 -mgeneral-regs-only -ffreestanding -nostdlib -nostartfiles -fno-ident -fno-asynchronous-unwind-tables -c "$cfile" -o "$ofile" || exit 1
done < "${link}/link64" || exit 1

output="${build}/kernel64.bin"
elf="${build}/kernel64.elf"
ld -m elf_x86_64 -T "${link}/ld64" --entry=_start "${o64[@]}" -o "$elf" || exit 1
objcopy -O binary -R .note.gnu.property -R .note.gnu.build-id "$elf" "$output"

if [ $(stat -c%s "$output") -gt 4096 ]; then # Change to whatever the number of sectors * 512 the bootloader is reading
    echo "${output} file is too large"
    exit 1
fi

dd "if=$output" "of=$image" bs=512 seek=9 conv=notrunc || exit 1

# Run it with 256MiB memory (-cpu qemu64,+smep)
if [[ "$1" != "--norun" ]]; then
    qemu-system-x86_64 -drive format=raw,file="$image" -m 256
    # qemu-system-x86_64 -device isa-serial -M pc -drive format=raw,file="$image" -m 256
fi
```

These files are used in the compilation of the kernel

link/ld32

```
SECTIONS {
    . = 0x1000;
    .text : { *(.text) }
    .data : { *(.data) }
    .bss : { *(.bss) }
    .eh_frame : { *(.eh_frame) }
    .got.plt : { *(.got.plt) }
    /* Place .note sections immediately after, no high addresses */
    .note : { *(.note*) }
    /* Discard unneeded sections */
    /DISCARD/ : { *(.comment .symtab .strtab .shstrtab) }
}
```

link/ld64

```
SECTIONS {
    . = 0x2000;
    .text : { *(.text) }
    .data : { *(.data) }
    .bss : { *(.bss) }
    .eh_frame : { *(.eh_frame) }
    .got.plt : { *(.got.plt) }
    /* Place .note sections immediately after, no high addresses */
    .note : { *(.note*) }
    /* Discard unneeded sections */
    /DISCARD/ : { *(.comment .symtab .strtab .shstrtab) }
}
```

link/link32

```
bootloader.c
identity.c
shared_constants.c
```

link/link32

```
kernel.c
shared_constants.c
utils.c
interrupts.c
```

The following files are the c, header and assembly files for the kernel
bootloader.asm

```
BITS 16 ; Produce 16 bit code
ORG 0x7c00 ; Set the origin to 0x7c00 which is where BIOS loads the bootloader

_start:
    jmp 0:.ClearCS ; Set CS to 0 (some bios load at 7C00:0000 instead of 0000:7C000)

.ClearCS:
    mov ax, 0xb800          ; Set AX to the video memory segment (0xb800)
    mov es, ax              ; Load ES with the video memory segment

    xor di, di              ; Start at the beginning of the video memory (offset 0)
    mov cx, 2000            ; 80 characters wide and 25 characters tall

clearscreen:
    mov byte [es:di], 0x20  ; Write character (space character) to video memory at ES:DI
    inc di                  ; Move to next byte (the attribute)

    mov byte [es:di], 0x07  ; Write attribute (light gray on black) to video memory at ES:DI
    inc di                  ; Move to the next byte (the character)

    dec cx                  ; Decrement CX
    jnz clearscreen         ; Jump to 'clearscreen' if CX is not zero

    lea si, [message]       ; Load the effective address of 'message' into SI
    xor di, di              ; Start at the beginning of the video memory (offset 0)
.loop:
    mov byte al, [si]

    cmp al, 0
    je read_kernel

```

```
    mov byte [es:di], al
    inc si
    add di, 2

    jmp .loop

error:
    mov ax, 0xb800          ; Set AX to the video memory segment (0xb800)
    mov es, ax              ; Load ES with the video memory segment
    xor di, di
    lea si, [error_message]
.loop:
    mov byte al, [si]

    cmp al, 0
    je halt

    mov byte [es:di], al
    inc si
    add di, 2

    jmp .loop

halt:
    cli
.loop:
    hlt
    jmp .loop

read_kernel:
    xor ax, ax      ; Clear AX
    mov ds, ax      ; Set DS to 0 (flat real-mode addressing)
    mov es, ax      ; Set ES to 0 (destination segment for read)

    mov ah, 0x02    ; BIOS function: read sectors
    mov al, sectors_read ; Number of sectors to read
    mov ch, 0       ; Cylinder 0
    mov cl, 2       ; Sector 2 (1-based indexing)
    mov dh, 0       ; Head 0
    mov dl, 0x80    ; Boot disk (0x80 = first hard disk, 0x00 = floppy)
    mov bx, 0x1000  ; Destination address ES:BX = 0x0:0x1000
    int 0x13        ; Call BIOS disk service

    jc error        ; If carry flag set, handle error
    cmp al, sectors_read ; Check if all sector were read
    jne error       ; If not, handle error

    ; setup gdt
    lea di, [gdt_location]
    lea si, [gdt_contents]
    ; first entry is the lgdt operand / null segment descriptor
    mov cx, gdt_size

setup_gdt_loop:
    jcxz enable_protection
    mov byte al, [si]
    mov byte [di], al
    inc si
    inc di
    dec cx
    jmp setup_gdt_loop

enable_protection:
    cli ; disable interrupts

    ; enable A20 (doesnt appear to be required)
    in al, 0x92
    or al, 2
    out 0x92, al

    ; in al, 0x92
    ; test al, 2
    ; jnz after
    ; or al, 2
    ; and al, 0xFE
    ; out 0x92, al
```

```
    ; after:

    lgdt [gdt_location] ; load the gdtr with the location of the gdt

    mov eax, cr0 ; load cr0 into eax
    or al, 1 ; set PE
    mov cr0, eax ; load eax back into cr0 (with PE enabled)
    jmp 08h:jump_32 ; far jump to callc

BITS 32
jump_32:
    mov ax, 10h
    mov ds, ax
    mov ss, ax
    mov es, ax
    mov fs, ax
    mov gs, ax
    mov esp, 0x10000

    call 1000h ; _start in C

    ; Enable PAE
    mov eax, cr4
    or eax, 0b100000
    mov cr4, eax

    ; Set CR3
    mov eax, 1 << 20
    mov cr3, eax

    ; Set IA32_EFER.LME
    mov ecx, 0xC0000080
    rdmsr
    or eax, 0x100
    wrmsr

    ; Enable Paging
    mov eax, cr0
    or eax, 1 << 31
    mov cr0, eax

    jmp 8h:jump_64

BITS 64
jump_64:
    mov ax, 10h
    mov ds, ax
    mov ss, ax
    mov es, ax
    mov fs, ax
    mov gs, ax
    mov rsp, 0x10000

    call 2000h ; _start in C
    jmp halt

message db 'Hello world', 0 ; The 'Hello World' message followed by a null terminator (0)
error_message db 'Failed to read kernel from disk', 0
gdt_location equ 0x0800 ; gdt location
gdt_contents:
    dq 0x000000000800001F
    dq 0x00CF9A000000FFFF
    dq 0x00CF92000000FFFF
    dq 0x0000000000000000
gdt_size equ $ - gdt_contents ; gdt location
sectors_read equ 0x10
; Boot sector padding and signature
times 510-($-$$) db 0   ; Pad the boot sector to 510 bytes (ensuring the total size is 512 bytes)
dw 0xAA55               ; Boot sector signature (0xAA55), required for a valid bootable sector
```

bootloader.c

```
#include <stdint.h>
#include "../include/identity.h"
#include "../include/shared_constants.h"

// Prints a new line to the vga text buffer
```

```c
void println(char* print);
uint32_t check_cpuid(void);

void _start() {
    println("Hello from 32-bit C running in protected mode");

    char* cpuid_str = "Does not suppport CPUID";
    if (check_cpuid()) {
        cpuid_str = "Suppports CPUID";
    }

    println(cpuid_str);

    paging_create();

    uint32_t cr3 = IDENTITY_BASE_ADDRESS;

    // Clear D/B for segment descriptors and set L
    uint64_t* segments = (uint64_t*)0x0808;
    const uint64_t SEG_FLAGS = ((uint64_t)1 << 54) | ((uint64_t)1 << 53);
    *segments ^= SEG_FLAGS;

    println("Idk");
}

void println(char* print) {
    static uint32_t line = 1;
    char volatile* vga = (char volatile*)(line * 160 + VGA_TEXT_BUFFER);
    while (*print != '\0') {
        *vga = *print;
        print++;
        vga += 2;
    }
    line++;
}

// Checks if CPUID is supported by attempting to flip the ID bit (bit 21) in EFLAGS.
// Returns 1 if CPUID is supported, 0 otherwise.
uint32_t check_cpuid(void) {
    uint32_t result;
    __asm__ volatile (
        "pushf\n\t"              // Save EFLAGS to stack
        "pop %%eax\n\t"           // Pop EFLAGS into eax
        "mov %%eax, %%ecx\n\t"    // Save original EFLAGS in ecx
        "xor $0x200000, %%eax\n\t"// Flip ID bit (1 << 21)
        "push %%eax\n\t"          // Push modified EFLAGS
        "popf\n\t"               // Load modified EFLAGS
        "pushf\n\t"              // Push EFLAGS again
        "pop %%eax\n\t"           // Pop into eax
        "push %%ecx\n\t"          // Restore original EFLAGS
        "popf\n\t"               // Load original EFLAGS
        "xor %%ecx, %%eax\n\t"    // Compare flipped vs original
        "jnz 1f\n\t"              // If different, CPUID supported
        "mov $0, %0\n\t"          // No CPUID, set result = 0
        "jmp 2f\n\t"
        "1:\n\t"
        "mov $1, %0\n\t"          // CPUID supported, set result = 1
        "2:\n\t"
        : "=r" (result)           // Output: result
        :                         // No inputs
        : "%eax", "%ecx", "cc"    // Clobbered: eax, ecx, condition codes
    );
    return result;
}
```

utils.h

```c
#pragma once

#include <stdint.h>

char* u64_to_string(int64_t integer);
char* i64_to_string(int64_t integer);
char println(char* string);

__attribute__((no_caller_saved_registers))
uint8_t inb(uint16_t port);
```

```
__attribute__((no_caller_saved_registers))
void outb(uint16_t port, uint8_t value);
```

utils.c

```
#include "../include/shared_constants.h"
#include <stdint.h>

char shared[64] = {};

void internal_u64_to_string(uint64_t integer, char* ptr) {
    // Store the string backwards
    uintptr_t length = 0;
    if (integer == 0) {
        ptr[0] = '0';
        length++;
    } else {
        while (integer > 0) {
            ptr[length] = (integer % 10) + '0';
            length++;
            integer = integer / 10;
        }
    }

    // Reverse the string
    for (uintptr_t i = 0; i < (length >> 1); i++) {
        uintptr_t j = length - i - 1;
        ptr[i] ^= ptr[j];
        ptr[j] ^= ptr[i];
        ptr[i] ^= ptr[j];
    }

    // Add a null terminator
    ptr[length] = 0;
}

char* u64_to_string(uint64_t integer) {
    internal_u64_to_string(integer, shared);
    return shared;
}

char* i64_to_string(int64_t integer) {
    if (integer < 0) {
        shared[0] = '-';
        internal_u64_to_string((uint64_t)-integer, shared + 1);
    } else {
        internal_u64_to_string((uint64_t)integer, shared);
    }

    return shared;
}

/// If it cannot print the entire string, it returns 1 else it returns 0
char println(char* string) {
    uint64_t* start = (uint64_t*)VGA_TEXT_BUFFER;
    const uintptr_t REP = ((VGA_HEIGHT - 2) * VGA_WIDTH * 2) / sizeof(uintptr_t);
    for (uint64_t i = 20; i < REP; i++) {
        start[i] = start[i + 20]; // 20 = 160/8
    }

    char* bytes = (char*)(start + REP);
    for (uint64_t i = 0; i < VGA_WIDTH; i++) {
        bytes[i << 1] = 0;
    }

    for (uint64_t i = 0; i < VGA_WIDTH; i++) {
        if (*string == '\0') {
            return 0;
        }

        bytes[i << 1] = *string;
        string++;
    }

    return *string != '\0';
}

```

```c
__attribute__((no_caller_saved_registers))
uint8_t inb(uint16_t port) {
    uint8_t value;
    __asm__ volatile (
        "inb %1, %0\n\t"
        : "=a"(value)
        : "Nd"(port)
        : "memory"
    );
    return value;
}

__attribute__((no_caller_saved_registers))
void outb(uint16_t port, uint8_t value) {
    __asm__ volatile (
        "outb %0, %1\n\t"
        :
        : "a"(value), "Nd"(port)
        : "memory"
    );
}
```

kernel.c

```c
/// 64 bit kernel

#include <stdint.h>
#include "../include/interrupts.h"
#include "../include/utils.h"
#include "../include/shared_constants.h"

void _start() {
    // Create enterline
    uint16_t* vga = (uint16_t*)(VGA_TEXT_BUFFER);
    for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
        vga[i] = 0x4F20;
    }

    vga += VGA_WIDTH * (VGA_HEIGHT - 1);
    for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
        vga[i] = 0x1F20;
    }

    println("64-bit mode");

    create_idt();

    // __asm__ volatile("int $0x80\n\t");

    while (1) {
        __asm__ volatile("hlt\n\t");
    }
}
```

identity.h

```c
#pragma once

void paging_create();
```

identity.c

```c
// Assumes 256MiB (28 bits) physical memory can be paged
// First MiB is not written to since there is often BIOS/UEFI data in it
// Uses level 4 paging

#include <stdint.h>
#include "../include/shared_constants.h"
#include "../include/identity.h"

const uint64_t FLAGS = 0b000000000011; // R/W, P

void create_pde(uint64_t* base) {
    uint64_t* pte_base = base + 512;
    for (uintptr_t i = 0; i < 128; i++) {
        *base = (uintptr_t)pte_base | FLAGS;
        base++;
```

```c
        pte_base += 512;
    }

    // Set remaining entries to 0
    for (uintptr_t i = 0; i < 384; i++) {
        *base = 0;
        base++;
    }

    // Creates 256 * 512 Pages
    uint64_t dst = 0;
    for (uintptr_t i = 0; i < 512 * 128; i++) {
        *base = dst | 3;
        base++;
        dst += 1 << 12;
    }
}

/// Only allocate first PDPTE and return its base address
/// base must be aligned to 4KiB and in a low address (eg 1MiB)
/// (also used to create pdpte since it has the same structure)
uint64_t* create_plm4e(uint64_t* base) {
    // Zero all other entries (P flag is 0)
    for (uintptr_t i = 1; i < 512; i++) {
        base[i] = 0;
    }

    uint64_t* pdpte = base + 512;
    *base = (uint64_t)pdpte | FLAGS;
    return pdpte;
}

void paging_create() {
    uint64_t* pdpte = create_plm4e((uint64_t*)IDENTITY_BASE_ADDRESS);
    uint64_t* pde = create_plm4e(pdpte);
    create_pde(pde);
}
```

interrupts.h

```c
#pragma once

void create_idt();
```

interrupts.c

```c
#include "../include/shared_constants.h"
#include "../include/utils.h"
#include <inttypes.h>
#include <stdint.h>

struct int_desc {
    uint16_t offset_1;        // offset bits 0..15
    uint16_t selector;        // a code segment selector in GDT or LDT
    uint8_t  ist;             // bits 0..2 holds Interrupt Stack Table offset, rest of bits zero.
    uint8_t  type_attributes; // gate type, dpl, and p fields
    uint16_t offset_2;        // offset bits 16..31
    uint32_t offset_3;        // offset bits 32..63
    uint32_t zero;            // reserved
};

struct int_frame {
    uintptr_t ip;
    uintptr_t cs;
    uintptr_t flags;
    uintptr_t sp;
    uintptr_t ss;
};

__attribute__((no_caller_saved_registers))
static void int_println(char* string) {
    uint64_t* start = (uint64_t*)VGA_TEXT_BUFFER;
    const uintptr_t REP = ((VGA_HEIGHT - 2) * VGA_WIDTH * 2) / sizeof(uint64_t);
    for (uintptr_t i = 20; i < REP; i++) {
        start[i] = start[i + 20];
    }

```

```
    char* bytes = (char*)(start + REP);
    for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
        bytes[i << 1] = 0;
    }

    for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
        if (*string == '\0') {
            return;
        }

        bytes[i << 1] = *string;
        string++;
    }
}

__attribute__((no_caller_saved_registers))
static void int_print_status(char* string) {
    uint16_t* start = (uint16_t*)VGA_TEXT_BUFFER;

    for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
        start[i] = 0x4F20;
    }

    char* bytes = (char*)VGA_TEXT_BUFFER;
    for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
        if (*string == '\0') {
            return;
        }

        bytes[i << 1] = *string;
        string++;
    }
}

__attribute__((interrupt))
static void default_handler(struct int_frame* frame) {
    int_print_status("Unknown Interrupt");
}

__attribute__((no_caller_saved_registers))
static char hex(uint8_t nibble) {
    if (nibble > 9) {
        return nibble + 'A' - 10;
    } else {
        return nibble + '0';
    }
}

__attribute__((no_caller_saved_registers))
static void update_cursor(uintptr_t x) {
    uint16_t pos = (VGA_HEIGHT - 1) * VGA_WIDTH + x;

    outb(0x3D4, 0x0F);
    outb(0x3D5, (uint8_t)(pos & 0xFF));
    outb(0x3D4, 0x0E);
    outb(0x3D5, (uint8_t)((pos >> 8) & 0xFF));
}

static char scancode_map[256] = {
    // 00
    '\0', '\0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '0', '-', '=', '\0', '\0',
    'q', 'w', 'e', 'r', 't', 'y', 'u', 'i', 'o', 'p', '[', ']', '\0', '\0', 'a', 's',
    'd', 'f', 'g', 'h', 'j', 'k', 'l', ';', '\'', '`', '\0', '#', 'z', 'x', 'c', 'v',
    'b', 'n', 'm', ',', '.', '/', '\0', '\0', '\0', ' ', '\0', '\0', '\0', '\0', '\0', '\0',
    // 40
    '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
    '\0', '\0', '\0', '\0', '\0', '\0', '\\', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
    '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
    '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
    // 80
    '\0', '\0', '!', '"', '\0', '$', '%', '^', '&', '*', '(', ')', '_', '+', '\0', '\0',
    'Q', 'W', 'E', 'R', 'T', 'Y', 'U', 'I', 'O', 'P', '{', '}', '\0', '\0', 'A', 'S',
    'D', 'F', 'G', 'H', 'J', 'K', 'L', ':', '@', '\0', '\0', '~', 'Z', 'X', 'C', 'V',
    'B', 'N', 'M', '<', '>', '?', '\0', '\0', '\0', ' ', '\0', '\0', '\0', '\0', '\0', '\0',
    // C0
    '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
```

```
    '\0', '\0', '\0', '\0', '\0', '\0', '|', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
    '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
    '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0',
};

__attribute__((interrupt))
static void keyboard_handler(struct int_frame* frame) {
    uint8_t scancode = inb(0x60);

    static uintptr_t char_index = 0;
    static char shift = 0;

    uint16_t* vga = (uint16_t*)(VGA_TEXT_BUFFER + VGA_WIDTH * 2 * (VGA_HEIGHT - 1));
    // for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
    //     vga[i] = 0x1F20;
    // }

    if (scancode < 0x80) {
        scancode |= shift;
        char map = scancode_map[scancode];
        if (map == '\0') {
            if (scancode == 0x2A) {
                shift = 0x80;
            } else if (scancode == 0x0E) {
                if (char_index > 0) {
                    char_index--;
                    *(char*)(vga + char_index) = 0x20;
                    update_cursor(char_index);
                }
            } else if (scancode == 0x1C) {
                int_println("");
                uint16_t* line = vga - VGA_WIDTH;
                for (uintptr_t i = 0; i < VGA_WIDTH; i++) {
                    line[i] = (vga[i] & 0xFF) | 0x0700;
                    vga[i] = 0x1F20;
                    char_index = 0;
                    update_cursor(char_index);
                }
            } else {
                char* string = "AA";
                string[0] = hex(scancode >> 4);
                string[1] = hex(scancode & 0xF);
                int_print_status(string);
            }
        } else {
            if (char_index < VGA_WIDTH) {
                *(char*)(vga + char_index) = map;
                char_index++;
                update_cursor(char_index);
            }
        }
    } else {
        if (scancode == 0xAA) {
            shift = 0;
        }
    }

    outb(0x20, 0x20);
}

__attribute__((interrupt))
static void int_handler(struct int_frame* frame) {
    int_print_status("Interrupt 80h");
}

static void create_entry(uint8_t entry, void (*ptr)(struct int_frame*), uint8_t type_attributes) {
    struct int_desc desc;
    desc.zero = 0;
    desc.offset_1 = (uint16_t)(uintptr_t)ptr;
    desc.offset_2 = (uint16_t)((uintptr_t)ptr >> 16);
    desc.offset_3 = (uint32_t)((uintptr_t)ptr >> 32);
    desc.ist = 0;
    desc.type_attributes = type_attributes;
    desc.selector = 0x08;
    ((struct int_desc*)IDT_BASE_ADDRESS)[entry] = desc;
}
```

```c

void create_idt() {
    // zero out all interrupt descriptors
    for (uint16_t i = 0; i < 256; i++) {
        create_entry((uint8_t)i, &default_handler, 0xEE);
    }

    create_entry(0x80, &int_handler, 0xEE);
    create_entry(0x21, &keyboard_handler, 0xEE);

    uint16_t* lidt = (uint16_t*)(IDT_BASE_ADDRESS - 10);
    *lidt = 256 * sizeof(struct int_desc) - 1;
    *(uint64_t*)(IDT_BASE_ADDRESS - 8) = IDT_BASE_ADDRESS;

    __asm__ volatile (
        "lidt %0\n\t"
        "sti\n\t"
        :
        : "m" (*lidt)
        : "memory"
    );

    // Set cursor to 0
    update_cursor(0);

    // Enable PIC
    outb(0x20, 0x11);  // Initialize the command port
    outb(0x21, 0x20);  // Set vector offset (IRQ0-IRQ7)
    outb(0x21, 0x04);  // Set cascading (IRQ2)
    outb(0x21, 0x01);  // Set 8086 mode
    outb(0x21, 0xFF);  // Mask all interrupts initially

    outb(0x21, inb(0x21) & 0xFD);  // Unmask IRQ1 (keyboard)
}
```

shared_constants.h

```c
#pragma once

#include <stdint.h>

extern const uintptr_t IDENTITY_BASE_ADDRESS;
extern const uintptr_t IDT_BASE_ADDRESS;

extern const uintptr_t VGA_TEXT_BUFFER;
extern const uintptr_t VGA_WIDTH;
extern const uintptr_t VGA_HEIGHT;
```

shared_constants.c

```c
#include "../include/shared_constants.h"
#include <stdint.h>

const uintptr_t IDENTITY_BASE_ADDRESS = 1 << 20;
const uintptr_t IDT_BASE_ADDRESS = 0x1750000;

const uintptr_t VGA_TEXT_BUFFER = 0xB8000;
const uintptr_t VGA_WIDTH = 80;
const uintptr_t VGA_HEIGHT = 25;
```

8 References

[1] *Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 1: Basic Architecture*, March 2025.

[2] *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes 2A, 2B, 2C, and 2D: Instruction Set Reference, A- Z*, March 2025.

[3] *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes 3A, 3B, 3C, and 3D: System Programming Guide*, March 2025.

[4] *Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 4: Model-specific Registers*, March 2025.

[5] Real mode.
"https://en.wikipedia.org/wiki/Real_mode". (accessed: 17/02/2026).

[6] Protected mode.
"https://en.wikipedia.org/wiki/Protected_mode". (accessed: 17/02/2026).

[7] Long mode.
"https://en.wikipedia.org/wiki/Long_mode". (accessed: 17/02/2026).

[8] Unreal mode.
"https://en.wikipedia.org/wiki/Unreal_mode". (accessed: 17/02/2026).

[9] System management mode.
"https://en.wikipedia.org/wiki/System_Management_Mode". (accessed: 17/02/2026).

[10] Virtual 8086 mode.
"https://en.wikipedia.org/wiki/Virtual_8086_mode". (accessed: 17/02/2026).

[11] Msi protocol.
"https://en.wikipedia.org/wiki/MSI_protocol". (accessed: 17/02/2026).

[12] Mesi protocol.
"https://en.wikipedia.org/wiki/MESI_protocol". (accessed: 17/02/2026).

[13] Moesi protocol.
"https://en.wikipedia.org/wiki/MOESI_protocol". (accessed: 17/02/2026).

[14] Mesif protocol.
"https://en.wikipedia.org/wiki/MESIF_protocol". (accessed: 17/02/2026).

[15] Cache placement policies.
"https://en.wikipedia.org/wiki/Cache_placement_policies". (accessed: 18/02/2026).

[16] Types of cache misses.
"https://www.geeksforgeeks.org/computer-organization-architecture/types-of-cache-misses/". (accessed: 18/02/2026).

[17] Intel x86 jump quick reference.
"http://unixwiz.net/techtips/x86-jumps.html". (accessed: 17/02/2026).

[18] Interrupts.
"https://wiki.osdev.org/Interrupts". (accessed: 01/03/2026).

[19] 8259 pic.
"https://wiki.osdev.org/8259_PIC". (accessed: 01/03/2026).

[20] syscall(2) — linux manual page.
"https://www.man7.org/linux/man-pages/man2/syscall.2.html". (accessed: 20/02/2026).

[21] 32-bit syscall numbers.
"https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/arch/x86/entry/syscalls/syscall_32.tbl". (accessed: 20/02/2026).

[22] 64-bit syscall numbers.
"https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/arch/x86/entry/syscalls/syscall_64.tbl". (accessed: 20/02/2026).

[23] read(2) - linux manual page.
"https://www.man7.org/linux/man-pages/man2/read.2.html". (accessed: 21/02/2026).

[24] write(2) - linux manual page.
"https://www.man7.org/linux/man-pages/man2/write.2.html". (accessed: 21/02/2026).

[25] open(2) - linux manual page.
"https://www.man7.org/linux/man-pages/man2/open.2.html". (accessed: 21/02/2026).

[26] close(2) - linux manual page.
"https://www.man7.org/linux/man-pages/man2/close.2.html". (accessed: 21/02/2026).

[27] stat(2) - linux manual page.
"https://www.man7.org/linux/man-pages/man2/lstat.2.html". (accessed: 21/02/2026).

[28] _exit(2) - linux manual page.
"https://www.man7.org/linux/man-pages/man2/_exit.2.html". (accessed: 21/02/2026).

[29] mmap(2) - linux manual page.
"https://www.man7.org/linux/man-pages/man2/mmap.2.html". (accessed: 21/02/2026).

[30] stat(3type) - linux manual page.
"https://www.man7.org/linux/man-pages/man3/stat.3type.html". (accessed: 21/02/2026).

[31] File descriptor.
"https://en.wikipedia.org/wiki/File_descriptor". (accessed: 21/02/2026).

9 Acknowledgements

Thank you to EMS for supporting this project, in particular:

- Dr Aeran Fleming
- Dr Ed Horncastle
- Michael Andrejczuk